

ALMA MATER STUDIORUM - UNIVERSITÀ DI BOLOGNA

Dipartimento di Informatica — Scienza e Ingegneria (DISI)
Corso di Laurea Magistrale in Ingegneria Informatica

CLEAR: Progetto di ricerca basato su Federated Learning per la rilevazione di Anomalie di Rete

**Tesi di Laurea Magistrale in
Intelligent Systems M**

Relatore:
Chiar.mo Prof. Michela Milano

Presentata da:
Roberto Di Giovanni

Correlatore:
Chiar.mo Prof. Andrea Borghesi

Anno Accademico 2025/2026

Indice

1	Introduzione	6
1.1	Scopo del Progetto	6
1.2	Il Lavoro Svolto	8
2	Schema Progettuale	10
2.1	Modello Ibrido	10
2.2	SNIDS	11
2.2.1	IDS	13
2.2.2	IPS	14
2.3	ADNIDS	15
2.4	Rappresentazione	16
3	Dataset	18
3.1	CIC	18
3.1.1	CIC-IDS2017	19
3.1.2	CSE-CIC-IDS2018	20
3.1.3	CIC-DDoS2019	23
3.1.4	CIC IoT-DIAD	24
3.1.5	Risultati Ottenuti	25
3.2	Log Files	27
3.2.1	Logstash vs Fluentd vs Fluent Bit	28
3.2.2	Come funziona Fluent Bit	30
3.2.3	Log Strutturato	32
3.2.4	GeoIP	34
3.2.5	CSV	36
3.2.6	Brute Force	38
4	Training	40
4.1	Data Preprocessing	40
4.2	Random Forest	43
4.3	Isolation Forest	45

4.4	HDBSCAN	48
4.5	Support Vector Machine	50
4.6	Autoencoder	51
4.7	Risultati Ottenuti	54
5	Federated Learning	56
5.1	Ciclo di vita del FL	56
5.2	Tipologie di FL	58
5.2.1	Distribuzione dei dati	58
5.2.2	Architettura di rete	58
5.2.3	Scala dei partecipanti	59
5.3	Script Python	59
5.4	Risultati Ottenuti	61
6	Sviluppi Futuri	63

Elenco delle figure

2.1	SNIDS vs ADNIDS	11
2.2	IDS ICMP	13
2.3	IPS ICMP	14
2.4	Modello Ibrido	16
3.1	Suddivisione del Dataset	18
3.2	CSE-CIC-IDS2018: Attacchi Eseguiti	22
3.3	CIC-DDoS2019: Attacchi Eseguiti	24
3.4	Accuracy dei modelli <i>CIC</i>	26
3.5	Logstash vs Fluentd vs Fluent Bit	29
3.6	Input	30
3.7	Parser	30
3.8	Filter	31
3.9	Buffer	31
3.10	Routing	32
3.11	Output	32
3.12	Ingest Pipeline di GeoIP	35
3.13	Mappa	35
3.14	CSV	37
4.1	Data Preprocessing applicato al CSV	43
4.2	Decision Tree	44
4.3	Punteggi delle <i>Features</i>	45
4.4	Isolation Forest	47
4.5	HDBSCAN Clustering	49
4.6	Support Vector Machine	50
4.7	Struttura di un Autoencoder	52
4.8	Distribuzione degli Errori di Ricostruzione	54
4.9	Metriche degli algoritmi	55
5.1	Round del <i>FL</i>	57
5.2	HFL vs VFL vs FTL	58

5.3	CFL vs DFL	59
5.4	Cross-Silo FL vs Cross-Device FL	59
5.5	Metriche Globali nei Round di <i>FL</i>	61

Capitolo 1

Introduzione

1.1 Scopo del Progetto

Al giorno d'oggi, la sicurezza informatica diventa sempre più importante e necessaria. Prendendo come casi di riferimento i recenti avvenimenti, i conflitti geopolitici evidenziano una totale convergenza tra operazioni cinetiche e cibernetiche. Per esempio, sul fronte russo-ucraino, la compromissione di oltre 10.000 telecamere di sorveglianza di rete per il tracciamento delle linee di rifornimento evidenzia come i dispositivi IoT standard siano diventati i punti d'accesso primari per l'esfiltrare informazioni tattiche. Diventa quindi fondamentale l'adozione di sistemi di Intelligenza Artificiale dedicati all'*Anomaly Detection*, capaci di analizzare costantemente la deviazione comportamentale dei flussi dati rispetto a una baseline di normalità.

Il progetto *CLEAR*, acronimo che sta per *Cybersecurity Laboratory for Edge Computing and Advanced Research*, è un progetto di ricerca sviluppato all'interno dell'azienda *Net Service* di Bologna in collaborazione con l'azienda *Flosslab*, nato dall'esigenza di portare una maggiore sicurezza e prevenzione dagli attacchi informatici.

Un attacco informatico (o *cyberattack*) è un'azione intenzionale e malevola condotta mediante computer, reti o dispositivi digitali per danneggiare, interrompere, alterare o accedere in modo non autorizzato a sistemi informatici, dati o infrastrutture.

Gli obiettivi per effettuarli possono essere molteplici fra cui:

- **Furto di dati:** Sottrarre credenziali, informazioni finanziarie, dati personali o segreti industriali;
- **Estorsione economica:** Bloccare siti web o server aziendali per chiedere un riscatto;
- **Spionaggio o sabotaggio:** Danneggiare la reputazione di un'entità o compromettere infrastrutture critiche.

La sfida del presente lavoro è definire un approccio per la mitigazione di tali minacce, strutturando un sistema in grado di rilevarle preventivamente.

La letteratura scientifica offre numerosi studi sull'analisi del traffico di rete e sull'*anomaly detection*, ovvero l'insieme di tecniche volte a identificare eventi o comportamenti che si discostano significativamente dal normale funzionamento. Attualmente, le soluzioni più avanzate per la risoluzione di questo problema si basano sull'impiego dell'intelligenza artificiale. Tra i contributi analizzati, di particolare rilievo risulta lo studio proposto da *Meryem Janati Idrissi, Hamza Alami, Abdelkader El Mahdaouy, Abdellah El Mekki, Soufiane Oualil, Zakaria Yartaoui e Ismail Berrada* (2023), intitolato "*Fed-ANIDS: Federated learning for anomaly-based network intrusion detection systems*" e pubblicato da *Elsevier*. Il lavoro presenta un innovativo sistema *Network Intrusion Detection System* (NIDS) basato su *Federated Learning* (FL) e modelli di *anomaly detection*.

Un NIDS è un sistema progettato per monitorare e analizzare il traffico di rete al fine di rilevare anomalie, intrusioni o violazioni. Utilizza approcci basati su firme (*signature-based*) e basati su anomalie (*anomaly-based*) per identificare e classificare i pattern di traffico come malevoli o benigni, con una preferenza per l'approccio basato su anomalie grazie alla sua capacità di rilevare attacchi *zero-day* e non identificati.

Il FL è un processo di *Machine Learning* (ML) distribuito che consente a più nodi di collaborare all'addestramento di un modello condiviso senza scambiarsi i dati grezzi. Offre diversi vantaggi chiave, quali la privacy dei dati, la sicurezza, l'efficienza e la scalabilità, mantenendo i dati a livello locale e scambiando solo gli aggiornamenti del modello attraverso la rete di comunicazione.

Lo studio di *Janati Idrissi et al.* (2023) si focalizza sullo sviluppo di un NIDS *anomaly-based*, scelto per la sua efficacia nel rilevare minacce non ancora note (*zero-day*). A tal fine, gli autori hanno implementato un modello di intelligenza artificiale basato su *Autoencoder*.

Questa specifica architettura di *Deep Learning* (DL) è ampiamente utilizzata nell'ambito dell'*anomaly detection* per la sua capacità di comprimere *pattern*

di dati complessi e ricostruirli quanto più fedelmente possibile alla forma originaria. In questo modo, un'eventuale discrepanza significativa tra il dato in ingresso e la sua ricostruzione in *output* segnala la presenza di un'anomalia. La sperimentazione si avvale dei *dataset* *USTC_TFC2016*, *CIC_IDS2017* e *CSE-CIC_IDS2018*, i quali condividono la medesima struttura: una componente relativa al traffico dati malevolo e una rappresentativa del traffico lecito. Tuttavia, l'addestramento tradizionale del modello mediante tali dataset richiederebbe la centralizzazione dell'intera mole di dati, ponendo evidenti criticità in termini di sicurezza e *privacy*; da qui nasce l'esigenza di adottare il *FL*. L'articolo in esame ha costituito il punto di partenza per il progetto *CLEAR*, incentrato sull'analisi delle tecniche di addestramento di modelli di intelligenza artificiale per la classificazione del traffico di rete benevolo o malevolo. Per garantire un elevato livello di riservatezza dei dati, il processo di addestramento adotta un approccio basato su *FL*.

1.2 Il Lavoro Svolto

Il contributo personale fornito all'interno del progetto ha riguardato diverse attività, ciascuna afferente a una specifica fase progettuale. Inizialmente è stata condotta un'analisi della letteratura scientifica al fine di individuare le possibili soluzioni al problema. Successivamente, lo studio si è focalizzato sulla valutazione di software *open source* per l'implementazione di sistemi *Signature-based NIDS* (*SNIDS*), tra cui *Snort*, *Suricata* e *Zeek*. L'analisi si è concentrata in particolare sugli ultimi due strumenti, testandoli sia in condizioni di traffico ordinario sia in scenari caratterizzati da attacchi di tipo *Denial of Service* (*DoS*) e *Brute Force*.

Un evento di *Denial of Service* (*DoS*) è un'azione cyber attack in cui gli hacker o i criminali informatici cercano di rendere una macchina host, un servizio online o una risorsa di rete non disponibile per gli utenti previsti.

Il *Brute Force Attack* è un metodo utilizzato dai cybercriminali per *craccare* le password degli account e scoprire credenziali di accesso. Questo tipo di attacco si basa su un dizionario di parole e password comuni che viene utilizzato per tentare di scoprire la password della vittima. Dopo aver esaurito tutti i termini presenti nel dizionario, i cybercriminali passano a tecniche più sofisticate, utilizzando combinazioni di caratteri, finché non viene trovata una corrispondenza.

A seguito dell'analisi comparativa dei software, la scelta è ricaduta su *Suricata*, in quanto integra funzionalità di *Intrusion Prevention System* (*IPS*), consentendo non solo di rilevare ma anche di bloccare le minacce note.

Nella fase successiva mi sono dedicato al processo di addestramento del modello, curando l'arricchimento della seconda tipologia di dataset. Tra gli algoritmi testati, ho provveduto all'addestramento del modello basato su *Autoencoder*, risultato il più prestazionale.

Infine, ho approfondito l'applicazione del *FL*, ottenendo risultati altamente soddisfacenti.

Le fasi progettuali sopra descritte verranno approfondite nei capitoli seguenti.

Capitolo 2

Schema Progettuale

2.1 Modello Ibrido

Nel precedente capitolo è stata discussa l'idea di partenza su cui si basa il progetto *CLEAR* (Janati Idrissi *et al.* (2023)). All'interno dell'articolo vengono citate due tipologie differenti di *NIDS*:

- **Signature-based NIDS (SNIDS):** sono basati su un database di attacchi noti, tramite il quale riescono a rilevare le intrusioni con un basso rate di falsi-positivi. Sono molto precisi per quanto riguarda le minacce conosciute, ma ciechi di fronte agli attacchi nuovi o variati;
- **Anomaly Detection-based NIDS (ADNIDS):** sono basati sul rilevare anomalie quando ci sono deviazioni dal comportamento del traffico normale, ma con un alto tasso di falsi-positivi. Questo vuol dire che qualsiasi comportamento di rete legittimo ma insolito (es. un picco improvviso di traffico per un backup straordinario o un aggiornamento software) viene scambiato per un attacco, generando allarmi inutili.

L'obiettivo del *paper* è la realizzazione di un sistema *ADNIDS*, in grado di sfruttare il vantaggio intrinseco di rilevare gli attacchi di tipo *zero-day*; tuttavia, tale approccio è caratterizzato da un tasso di falsi positivi superiore rispetto a quello dei sistemi *SNIDS*. Per questo motivo, l'indagine condotta nell'ambito del progetto *CLEAR* si è ampliata considerando anche l'integrazione delle metodologie *SNIDS*.

Estendendo la ricerca in tale direzione, è stata presa in esame la strategia proposta in letteratura da Said Ouiazzane, Malika Addou e Fatimazahra Barramou (2022), nel contributo intitolato "*A Suricata and Machine Learning Based Hybrid Network Intrusion Detection System*" e pubblicato da Springer, all'interno del libro/volume di atti di convegno *Advances in Information, Communication*

and Cybersecurity, appartenente alla collana *Lecture Notes in Networks and Systems*, vol. 357. Tale approccio prevede la progettazione di un NIDS di tipo ibrido, che combina le architetture SNIDS e ADNIDS.

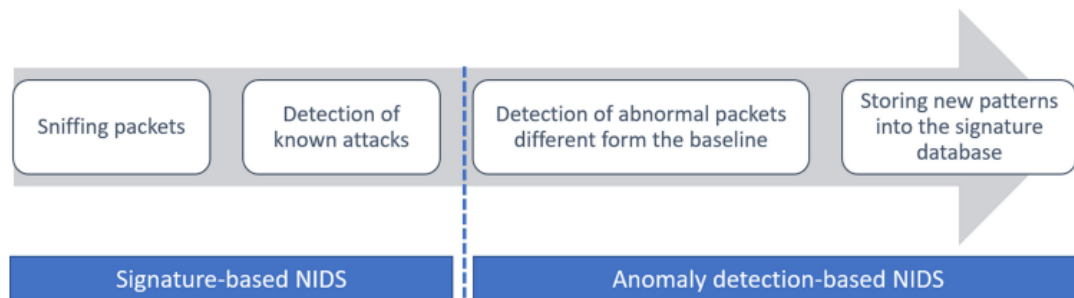


Figura 2.1: SNIDS vs ADNIDS

Il principio sottostante si basa sulla complementarità delle due tecnologie: i punti di forza dell'una consentono di compensare i limiti intrinseci dell'altra, e viceversa.

2.2 SNIDS

Fra i software open source più famosi a ricoprire il ruolo di SNIDS, abbiamo *Snort*, *Suricata* e *Zeek*.

- **Snort:** il suo scopo è quello di analizzare i pacchetti di dati in transito confrontandoli con un database di regole di sicurezza. Se un pacchetto corrisponde al pattern di un attacco noto, *Snort* reagisce immediatamente. Ha tre utilizzi principali: come packet sniffer, intercetta e mostra a schermo il traffico di rete in tempo reale, come packet logger, registra il traffico su disco per successive analisi forensi o debugging di rete, o come vero e proprio sistema di prevenzione delle intrusioni di rete.
- **Suricata:** sviluppato dalla *Open Information Security Foundation (OISF)*, presenta una struttura simile a *Snort*, ma con qualche differenza. La sua architettura è *multi-threaded* in contrasto con quella *single-threaded* di *Snort*, che limita la cattura dei pacchetti in ambienti con alto traffico. *Suricata* supporta *X-Forwarded-For (XFF)*, il quale permette ai server proxy di condividere informazioni sui client originari e gli altri proxy nella catena, in modo da determinare, ad esempio, chi sta tentando l'accesso ad un sito web. *Suricata* può essere integrato facilmente con lo stack *ELK* e *Graylog*, è pienamente compatibile con le regole *Snort*, oltre a supportare regole avanzate proprie, ma richiede un consumo maggiore di risorse.

- **Zeek:** analizzatore di traffico di rete passivo indicato per supportare le indagini su attività sospette o malevole. Supporta inoltre un'ampia gamma di attività di analisi del traffico che vanno oltre l'ambito della sicurezza, tra cui la misurazione delle prestazioni e la risoluzione dei problemi. Il primo beneficio che un nuovo utente trae da *Zeek* è l'ampia serie di *log* che descrivono l'attività di rete. Questi *log* includono non solo una registrazione dettagliata di ogni connessione intercettata, ma anche trascrizioni a livello applicativo. Non è ottimizzato per il *byte-matching*, e gli utenti che cercano approcci basati sul *signature detection* farebbero meglio a provare sistemi di *intrusion detection (IDS)* come *Snort* e *Suricata*. Un *IDS* è un dispositivo hardware o un'applicazione software che monitora passivamente una rete o i sistemi informatici alla ricerca di attività dannose o violazioni delle policy di sicurezza e genera avvisi quando rileva potenziali minacce.

Essendo *Suricata* un'evoluzione di *Snort*, quest'ultimo non è stato approfondito ulteriormente lasciando spazio al suo successore e a *Zeek*. Ponendoci in un ambiente di testing, entrambi hanno fornito i risultati attesi.

La differenza principale fra i due è che *Zeek* non dispone della modalità *IPS* e, per questo motivo, la scelta è ricaduta su *Suricata* nel ruolo di *SNIDS*.

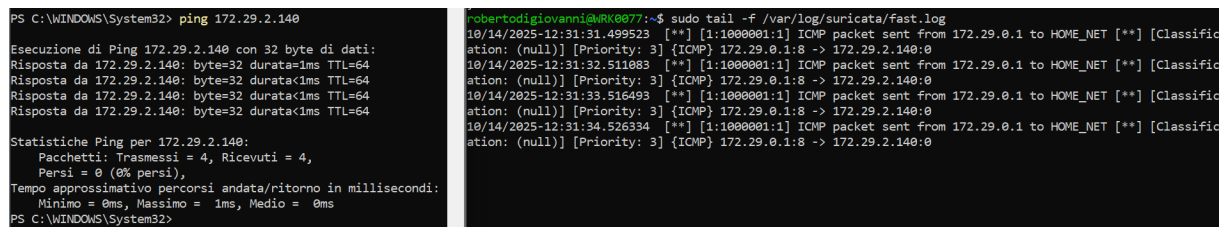
Un *IPS* è un dispositivo di sicurezza informatica (hardware o software) progettato per esaminare il traffico di rete e bloccare attivamente le minacce in tempo reale. Quando l'*IPS* individua qualcosa di malevolo, può reagire all'istante in diversi modi:

- **Bloccare i pacchetti:** scarta immediatamente i dati pericolosi, impedendo loro di raggiungere la destinazione;
- **Chiudere la connessione:** interrompe bruscamente la sessione di comunicazione tra l'attaccante e la vittima;
- **Isolare l'indirizzo IP:** blocca temporaneamente o permanentemente tutto il traffico proveniente dall'IP dell'aggressore;
- **Ripulire il traffico:** in alcuni casi, rimuove la parte infetta o malevola dal pacchetto dati, lasciando passare il resto del traffico legittimo.

Mentre un firewall chiude le porte d'accesso non autorizzate, l'*IPS* è fondamentale perché protegge dai cosiddetti exploit, ovvero quei codici malevoli che sfruttano i bug (le vulnerabilità) dei software legittimi.

2.2.1 IDS

Suricata cerca schemi pericolosi noti usando liste di regole. Per capire meglio il suo funzionamento nella modalità *IDS*, di seguito viene fornito un esempio di come interagiscono delle specifiche regole alla ricezione di un determinato evento.



```

PS C:\WINDOWS\System32> ping 172.29.2.140

Esecuzione di Ping 172.29.2.140 con 32 byte di dati:
Risposta da 172.29.2.140: byte=32 durata<1ms TTL=64
Risposta da 172.29.2.140: byte=32 durata<1ms TTL=64
Risposta da 172.29.2.140: byte=32 durata<1ms TTL=64
Risposta da 172.29.2.140: byte=32 durata<1ms TTL=64

Statistiche Ping per 172.29.2.140:
    Pacchetti: Trasmessi = 4, Ricevuti = 4,
    Persi = 0 (0% persi),
    Tempo approssimativo percorsi andata/ritorno in millisecondi:
        Minimo = 0ms, Massimo = 1ms, Medio = 0ms
PS C:\WINDOWS\System32>
  
```

```

robertodigiovanni@RK0077:~$ sudo tail -f /var/log/suricata/fast.log
10/14/2025-12:31:31.499523 [**] [1:1000001:1] ICMP packet sent from 172.29.0.1 to HOME_NET [**] [Classification: (null)] [Priority: 3] {ICMP} 172.29.0.1:8 -> 172.29.2.140:0
10/14/2025-12:31:32.511083 [**] [1:1000001:1] ICMP packet sent from 172.29.0.1 to HOME_NET [**] [Classification: (null)] [Priority: 3] {ICMP} 172.29.0.1:8 -> 172.29.2.140:0
10/14/2025-12:31:33.516493 [**] [1:1000001:1] ICMP packet sent from 172.29.0.1 to HOME_NET [**] [Classification: (null)] [Priority: 3] {ICMP} 172.29.0.1:8 -> 172.29.2.140:0
10/14/2025-12:31:34.526334 [**] [1:1000001:1] ICMP packet sent from 172.29.0.1 to HOME_NET [**] [Classification: (null)] [Priority: 3] {ICMP} 172.29.0.1:8 -> 172.29.2.140:0
  
```

Figura 2.2: IDS ICMP

Tramite la regola:

```
alert icmp 172.29.0.1 any -> $HOME_NET any (msg:"ICMP packet sent from 172.29.0.1 to HOME_NET"; itype:8; sid:1000001; rev:1;)
```

Dico a *Suricata* di avvisarmi quando arriva un pacchetto ICMP dall'indirizzo 172.29.0.1 con qualsiasi porta di origine fino all'indirizzo specificato dalla variabile *HOME_NET* avente qualsiasi porta di destinazione.

Quando il ping viene inviato, si riceve il messaggio contenuto all'interno di *msg*. *itype*, con valore 8, indica che vengono filtrati i pacchetti ICMP di tipo *Echo Request*.

il *sid* è un identificatore numerico univoco per la regola. I valori da 1.000.000 in poi sono riservati all'uso personalizzato/locale.

rev rappresenta la versione della regola. Serve a tracciare quante volte è stata modificata questa specifica regola.

2.2.2 IPS

Nell'immagine sottostante, si può osservare come *Suricata* operi in modalità *IPS*, utilizzando sempre la stessa regola, con la differenza fondamentale per il *drop* iniziale, il quale andrà a scartare i pacchetti quando si presenterà la specifica condizione.

The screenshot shows a terminal window with the following content:

```

robertodigiovanni@WRK0077:~$ sudo tail -f /var/log/suricata/fast.log
10/14/2025-16:52:33.793049 [Drop] [**] [1:1000004:1] ICMP packet blocked from 172.29.0.1 to HOME_NET [**] [Class
ification: (null)] [Priority: 3] {ICMP} 172.29.0.1:8 -> 172.29.2.140:0
10/14/2025-16:52:33.792819 [wDrop] [**] [1:1000004:1] ICMP packet blocked from 172.29.0.1 to HOME_NET [**] [Class
ification: (null)] [Priority: 3] {ICMP} 172.29.0.1:8 -> 172.29.2.140:0
10/14/2025-16:52:38.464457 [Drop] [**] [1:1000004:1] ICMP packet blocked from 172.29.0.1 to HOME_NET [**] [Class
ification: (null)] [Priority: 3] {ICMP} 172.29.0.1:8 -> 172.29.2.140:0
10/14/2025-16:52:38.464288 [wDrop] [**] [1:1000004:1] ICMP packet blocked from 172.29.0.1 to HOME_NET [**] [Class
ification: (null)] [Priority: 3] {ICMP} 172.29.0.1:8 -> 172.29.2.140:0
10/14/2025-16:52:43.454312 [Drop] [**] [1:1000004:1] ICMP packet blocked from 172.29.0.1 to HOME_NET [**] [Class
ification: (null)] [Priority: 3] {ICMP} 172.29.0.1:8 -> 172.29.2.140:0
10/14/2025-16:52:43.454198 [wDrop] [**] [1:1000004:1] ICMP packet blocked from 172.29.0.1 to HOME_NET [**] [Class
ification: (null)] [Priority: 3] {ICMP} 172.29.0.1:8 -> 172.29.2.140:0
10/14/2025-16:52:47.934930 [Drop] [**] [1:1000004:1] ICMP packet blocked from 172.29.0.1 to HOME_NET [**] [Class
ification: (null)] [Priority: 3] {ICMP} 172.29.0.1:8 -> 172.29.2.140:0
10/14/2025-16:52:47.934799 [wDrop] [**] [1:1000004:1] ICMP packet blocked from 172.29.0.1 to HOME_NET [**] [Class
ification: (null)] [Priority: 3] {ICMP} 172.29.0.1:8 -> 172.29.2.140:0

robertodigiovanni@WRK0077:~$ sudo suricata -c /etc/suricata/suricata.yaml -q 0
i: suricata: This is Suricata version 8.0.1 RELEASE running in SYSTEM mode
i: mm-hs: Rule group caching - loaded: 112 newly cached: 0 total cacheable: 112
i: threads: Threads created -> W: 1 FM: 1 FR: 1 Engine started.
i: suricata: Signal Received. Stopping engine.
i: nfr: (W-NFQW) Treated: Pkts 14, Bytes 2306, Errors 0
i: nfr: (W-NFQW) Verdict: Accepted 10, Dropped 4, Replaced 0
  
```

On the right side of the terminal, there is a window showing the output of a ping command:

```

Statistiche Ping per 172.29.2.140:
Pacchetti: Trasmessi = 4, Ricevuti = 0,
Persi = 4 (100% persi),
PS C:\WINDOWS\System32> ping 172.29.2.140

Esecuzione di Ping 172.29.2.140 con 32 byte di dati:
Richiesta scaduta.
Richiesta scaduta.
Richiesta scaduta.
Richiesta scaduta.

Statistiche Ping per 172.29.2.140:
Pacchetti: Trasmessi = 4, Ricevuti = 0,
Persi = 4 (100% persi),
PS C:\WINDOWS\System32>
  
```

Figura 2.3: IPS ICMP

la regola è la seguente:

```
drop icmp 172.29.0.1 any -> $HOME_NET any (msg:"ICMP packet blocked from
172.29.0.1 to HOME_NET"; itype:8; sid:1000004; rev:1;)
```

Per testare l'efficacia della modalità *IPS*, è stata predisposta una semplice pagina *HTML* in ambiente locale, verso la quale è stato simulato un attacco di tipo *Brute Force* basato su un dizionario di password. Durante l'esecuzione dello script di attacco, viene generato un elevato volume di richieste al secondo indirizzate all'endpoint */login*. Mentre l'obiettivo della simulazione d'attacco è quello di ottenere l'accesso non autorizzato con le credenziali di un altro utente, il compito di *Suricata* è proteggere il sistema identificando e bloccando tempestivamente tali chiamate. Per ottenere tale risultato occorre impostare la seguente regola:

```
drop http any any -> 127.0.0.1 8000 (msg:"Brute Force Attempt - Login Reque-
st to /login"; flow:to_server, established; http.method; content:"POST"; http.uri;
content:"/login"; threshold: type threshold, track by_src, count 5, seconds 1; no-
case; classtype:bad-unknown; sid:1000008; rev:1;)
```

Viene utilizzato il campo *http* per ispezionare il traffico a livello applicativo *HTTP* (L7).

flow:to_server, established fa sì che venga controllato solo il traffico diretto verso il server e si attiva unicamente per connessioni *TCP* già stabilite.

http.method; content:"POST" ispeziona il metodo della richiesta *HTTP*. Cerca specificamente le richieste con metodo *POST*.

http.uri; content: "/login" ispeziona l'URI della richiesta *HTTP*, in particolare cerca il percorso */login*.

threshold: type threshold, track by_src, count 5, seconds 1 fa scattare la regola solo al raggiungimento di 5 tentativi di login nell'arco di 1 secondo dallo stesso indirizzo *IP*. Il conteggio viene effettuato separatamente per ogni singolo indirizzo *IP*.

nocase rende la ricerca del contenuto non sensibile al maiuscolo/minuscolo.

classtype:bad-unknown classifica la gravità e il tipo di minaccia nei sistemi *SIEM/log*.

2.3 ADNIDS

Terminata la precedente parte, si è data più enfasi allo studio dell'*ADNIDS*, essendo l'obiettivo primario della ricerca, lasciando temporaneamente in sospeso lo studio riguardante l'*SNIDS*.

L'*ADNIDS* in questione è il modello di Intelligenza Artificiale addestrato per riconoscere e distinguere il traffico dati benevolo da quello malevolo.

I sistemi tradizionali di prevenzione, come i firewall, non sono più sufficienti per annullare il cybercrime, dato che possono fermare attacchi conosciuti dall'esterno, ma non quelli dall'interno o quelli che sfruttano le vulnerabilità di zero-days. Per questo motivo, quando si presenta una variazione del flusso normale dei dati, il modello lo segnala come un tentativo di attacco.

Per la progettazione, lo sviluppo e l'addestramento del modello è stato adottato *Python*, affermatosi ormai come il linguaggio di programmazione di riferimento nell'ambito dell'Intelligenza Artificiale.

La scelta del dataset è importante e deve essere seguita da una fase di *Pre-processing* per convertire i dati grezzi in una matrice di numeri comprensibile dagli algoritmi di Intelligenza Artificiale, tramite un insieme di trasformazioni matematiche e logiche.

- **Step 2:** a questo punto, lo *Sniffer* ha catturato i pacchetti di rete ed è momento di verificarne la corrispondenza rispetto al database delle firme dello *SNIDS*. Quest'ultimo controlla la presenza di un'eventuale corrispondenza tra le caratteristiche del pacchetto catturato ed i pattern di intrusione noti memorizzati nel proprio database. Si possono distinguere due casi:
 - Se il pacchetto corrisponde ad un pattern riconosciuto dal database dello *SNIDS*, viene inviato un allarme all'amministratore di sicurezza per segnalare l'intrusione rilevata;
 - Se il pacchetto non corrisponde ad alcuna firma di attacco nota, esso deve passare al secondo componente, ovvero l'*ADNIDS*.
- **Step 3:** se il pacchetto di rete corrisponde ad un'intrusione nota, viene inviato automaticamente un'allarme agli amministratori di sicurezza affinché analizzino e indaghino sui pacchetti sospetti;
- **Step 4:** se il pacchetto di rete non viene riconosciuto come un'intrusione nota, interviene l'*ADNIDS* per confrontare le caratteristiche del pacchetto con la baseline di rete;
- **Step 5:** se le caratteristiche del pacchetto di rete deviano dalla baseline, viene inviato un allarme all'amministratore per informarlo della presenza di un pacchetto sospetto nel traffico di rete;
- **Step 6:** quando il pacchetto devia dalla baseline, viene considerato sospetto e la sua firma viene inviata allo *SNIDS*, in modo che il pacchetto possa essere riconosciuto come tale nei successivi rilevamenti;
- **Step 7:** se il pacchetto corrisponde ad un comportamento normale, viene considerato ordinario e non rappresenta un rischio di intrusione per gli asset informatici.

Capitolo 3

Dataset

Il seguente capitolo analizza l'evoluzione del dataset impiegato nella fase di addestramento, ripercorrendo il processo dai primi esperimenti fino al raggiungimento di una configurazione ottimale per il caso di studio.

3.1 CIC

Assumendo come riferimento lo studio condotto da *Janati Idrissi et al.*, richiamato nel primo capitolo, sono stati presi in considerazione i dataset forniti dal *Canadian Institute for Cybersecurity (CIC)*.

Il *CIC* è un'unità multidisciplinare e integrata che si occupa di formazione, ricerca e sviluppo, e imprenditorialità, avvalendosi delle competenze di ricercatori nei settori delle scienze sociali, dell'economia, dell'informatica, dell'ingegneria, del diritto e delle scienze naturali.

In continuità con tale approccio, si è scelto di analizzare tali basi dati, estendendo al contempo l'indagine verso le *release* più recenti sviluppate dal medesimo istituto.

Dataset	Features	Attacchi
CIC-IDS-Collection	Più di 80	Botnet, Brute-Force, DoS, DDoS, Infiltration, Portscan e Webattack
CIC IoT-DIAD	86	DDoS, DoS, Recon, Web-based, Brute Force, Spoofing, e Mirai

Figura 3.1: Suddivisione del Dataset

La tabella sovrastante, mostra il dataset *CIC-IDS-Collection*, in cui all'interno sono contenuti i dataset: *CIC-IDS2017*, *CSE-CIC-IDS2018* e *CIC-DDoS2019*, se-

guito dal dataset *CIC IoT-DIAD*. Ad ognuno di questi è associato il numero di *feature* presenti al loro interno, seguito dagli attacchi utilizzati che contraddistinguono i dataset.

3.1.1 CIC-IDS2017

Il dataset *CICIDS2017* contiene traffico benigno e i più recenti attacchi comuni, riflettendo la struttura dei dati nel mondo reale (file *PCAP*). Include inoltre i risultati dell'analisi del traffico di rete effettuata tramite *CICFlowMeter*, con flussi etichettati in base al *timestamp*, *IP di origine e destinazione*, *porte di origine e destinazione*, *protocolli e tipo di attacco* (file *CSV*).

È stato impiegato il sistema *B-Profile* per profilare il comportamento astratto delle interazioni umane e generare un traffico benigno naturale. Per questo dataset è stato modellato il comportamento astratto di 25 utenti basandosi sui protocolli *HTTP*, *HTTPS*, *FTP*, *SSH* ed *email*.

Il periodo di acquisizione dei dati è iniziato alle ore 9:00 di lunedì 3 luglio 2017 e si è concluso alle ore 17:00 di venerdì 7 luglio 2017, per un totale di 5 giorni. Il lunedì è la giornata di attività normale e include esclusivamente traffico benigno. Gli attacchi implementati comprendono *Brute Force FTP*, *Brute Force SSH*, *DoS*, *Heartbleed*, *Web Attack*, *Infiltration*, *Botnet* e *DDoS*, eseguiti sia di mattina che di pomeriggio nei giorni di martedì, mercoledì, giovedì e venerdì. La divisione è la seguente:

- **Lunedì:** Attività normale, 11GB
- **Martedì:**
 - *Attacchi:* FTP-Patator (9:20-10:20), SSH-Patator (14:00-15:00)
 - *Attaccante:* Kali, 205.174.165.73
 - *Vittima:* WebServer Ubuntu, 205.174.165.68 (Local IP: 192.168.10.50)
- **Mercoledì:**
 - *Attacchi:* DoS slowloris (9:47-10:10), DoS Slowhttptest (10:14-10:35) DoS Hulk (10:43-11), DoS GoldenEye (11:10-11:23)
 - *Attaccante:* Kali, 205.174.165.73
 - *Vittima:* WebServer Ubuntu, 205.174.165.68 (Local IP: 192.168.10.50)

- **Giovedì:**

- *Attacchi:* Web Attack-Brute Force (9:20-10), Web Attack-XSS (10:15-10:35), Web Attack-Sql Injection (10:40-10:42), Meta exploit Win Vista (14:19 e 14:20-14:21) e (14:33-14:35), Infiltration-Cool disk-MAC (14:53-15:00), Win Vista (15:04-15:45)
- *Attaccante:* Kali, 205.174.165.73
- *Vittima:* WebServer Ubuntu, 205.174.165.68 (Local IP: 192.168.10.50) Windows Vista, 192.168.10.8, MAC, 192.168.10.25

- **Venerdì:**

- *Attacchi:* Botnet ARES (10:02-11:02), DDoS LOIT (15:56-16:16)
- *Attaccante:* Kali, 205.174.165.73, Three Win 8.1, 205.174.165.69 - 71
- *Vittima:* Win 10, 192.168.10.15 + Win 7, 192.168.10.9 + Win 10, 192.168.10.14 + Win 8, 192.168.10.5 + Vista, 192.168.10.8, Ubuntu16, 205.174.165.68 (Local IP: 192.168.10.50)

3.1.2 CSE-CIC-IDS2018

Il dataset *CSE-CIC-IDS2018* è un progetto collaborativo fra il *Communications Security Establishment (CSE)* ed il *Canadian Institute for Cybersecurity (CIC)*, in cui all'interno viene utilizzato il concetto di profili per generare dataset in modo sistematico, contenenti descrizioni dettagliate delle intrusioni e modelli di distribuzione astratti per applicazioni, protocolli o entità di rete di livello inferiore. Questi profili possono essere impiegati da agenti o operatori umani per generare eventi sulla rete. Grazie alla natura astratta dei profili generati, è possibile applicarli ad un'ampia gamma di protocolli di rete con topologie differenti, oppure combinarli per creare un dataset adatto a esigenze specifiche. Definiremo due classi distinte di profili:

- **B-Profiles (Profili Benigni):** incapsulano i comportamenti delle entità utente utilizzando diverse tecniche di apprendimento automatico e analisi statistica (come *K-Means*, *Random Forest*, *SVM* e *J48*). Le caratteristiche incapsulate includono la distribuzione della dimensione dei pacchetti di un protocollo, il numero di pacchetti per flusso, specifici pattern nel payload, la dimensione del payload e la distribuzione dei tempi di richiesta di un protocollo. Nell'ambiente di test verranno simulati i seguenti protocolli: *HTTPS*, *HTTP*, *SMTP*, *POP3*, *IMAP*, *SSH* e *FTP*. In base alle nostre

osservazioni iniziali, la maggior parte del traffico è costituita da *HTTP* e *HTTPS*.

- **M-Profiles:** tentano di descrivere uno scenario di attacco in modo univoco e non ambiguo. Nel caso più semplice, gli operatori umani possono interpretare questi profili ed eseguirli manualmente. Idealmente, per l'interpretazione e l'esecuzione di tali scenari verrebbero impiegati agenti autonomi affiancati da compilatori. Per gli attacchi sono stati presi in considerazione sei diversi scenari:
 - *Infiltrazione nella rete dall'interno:* in questo scenario viene inviato un file malevolo tramite e-mail alla vittima per sfruttare una vulnerabilità applicativa. In seguito, viene eseguita una backdoor sul computer della vittima; successivamente viene utilizzata tale macchina per scansionare la rete interna alla ricerca di altri sistemi vulnerabili ed effettuare ulteriori exploit, ove possibile.
 - *Denial of Service HTTP:* per il seguente attacco viene utilizzata *Slowloris* e *LOIC* come strumenti principali, i quali hanno dimostrato di poter rendere i Web server completamente inaccessibili impiegando una singola macchina attaccante. *Slowloris* inizia stabilendo una connessione *TCP* completa verso il server remoto e la mantiene aperta inviando a intervalli regolari richieste *HTTP* valide ma incomplete, così da impedire la chiusura dei socket. Poiché ogni Web server dispone di una capacità limitata di gestione delle connessioni, è solo questione di tempo prima che vengano utilizzati tutti i socket e sia impossibile stabilire nuove connessioni. Un'altra celebre applicazione per lanciare attacchi DoS contro siti Web è *HOIC*.
 - *Insieme di attacchi ad applicazioni Web:* viene utilizzata la *Damn Vulnerable Web App (DVWA)*, sviluppata per aiutare i professionisti della sicurezza a testare le proprie competenze. Nella prima fase viene scansionato il sito Web tramite uno scanner di vulnerabilità; successivamente vengono condotti diversi tipi di attacchi Web sul sito vulnerabile, inclusi SQL injection, command injection ed unrestricted file upload.
 - *Attacchi di Brute force:* gli attacchi di *Brute Force* sono molto comuni contro le reti, in quanto puntano a violare account protetti da combinazioni deboli di nome utente e password. L'ultimo scenario è stato progettato con l'obiettivo di compromettere un account *SSH* ed un da-

tabase *MySQL* eseguendo un attacco *Brute Force* basato su dizionario contro il server principale.

- *Attacchi basati su vulnerabilità recenti:* esistono attacchi basati su vulnerabilità note che possono essere condotti in un intervallo di tempo specifico (si tratta di vulnerabilità eccezionali che talvolta colpiscono milioni di server o vittime e la cui risoluzione richiede solitamente mesi per aggiornare tutti i computer vulnerabili nel mondo).

Attack	Tools	Duration	Attacker	Victim
Bruteforce attack	FTP – Patator	One day	Kali linux	Ubuntu 16.4 (Web Server)
	SSH – Patator			
DoS attack	Hulk, GoldenEye,	One day	Kali linux	Ubuntu 16.4 (Apache)
	Slowloris, Slowhttptest			
DoS attack	Heartleech	One day	Kali linux	Ubuntu 12.04 (Open SSL)
Web attack	- Damn Vulnerable Web App (DVWA) - In-house selenium framework (XSS and Brute-force)	Two days	Kali linux	Ubuntu 16.4 (Web Server)
Infiltration attack	- First level: Dropbox download in a windows machine - Second Level: Nmap and portscan	Two days	Kali linux	Windows Vista and Macintosh
Botnet attack	- Ares (developed by Python): remote shell, file upload/download, capturing - screenshots and key logging	One day	Kali linux	Windows Vista, 7, 8.1, 10 (32-bit) and 10 (64-bit)
DDoS+PortScan	Low Orbit Ion Canon (LOIC) for UDP, TCP, or HTTP requests	Two days	Kali linux	Windows Vista, 7, 8.1, 10 (32-bit) and 10 (64-bit)

Figura 3.2: CSE-CIC-IDS2018: Attacchi Eseguiti

3.1.3 CIC-DDoS2019

Il dataset *CIC-DDoS2019* contiene traffico benigno e i più recenti attacchi *DDoS* comuni, riflettendo la struttura dei dati reali del mondo reale (file *PCAP*). Include inoltre i risultati dell'analisi del traffico di rete effettuata tramite *CICFlowMeter-V3*, con flussi etichettati in base al timestamp, IP di origine e destinazione, porte di origine e destinazione, protocolli e tipo di attacco (file *CSV*).

CICFlowMeter è un generatore e analizzatore di flussi di traffico di rete. Può essere utilizzato per generare flussi bidirezionali, in cui il primo pacchetto determina il *forward* (da sorgente a destinazione) ed il *backward* (da destinazione a sorgente); in questo modo è possibile calcolare separatamente per le due direzioni oltre 80 metriche statistiche del traffico, come durata, numero di pacchetti, numero di byte, lunghezza dei pacchetti, ecc.

La generazione di un traffico di sottofondo realistico ha rappresentato la massima priorità nella creazione di questo dataset. È stato impiegato il sistema *B-Profile* proposto per profilare il comportamento astratto delle interazioni umane e generare un traffico di sottofondo benigno nell'ambiente di test proposto. Per questo dataset è stato modellato il comportamento astratto di 25 utenti basandosi sui protocolli *HTTP*, *HTTPS*, *FTP*, *SSH* ed *email*.

Come mostrato nella Tabella sottostante, sono stati eseguiti 12 attacchi *DDoS* (tra cui *NTP*, *DNS*, *LDAP*, *MSSQL*, *NetBIOS*, *SNMP*, *SSDP*, *UDP*, *UDP-Lag*, *WebD-DoS*, *SYN* e *TFTP*) nella giornata dedicata all'addestramento, e 7 attacchi (tra cui *PortScan*, *NetBIOS*, *LDAP*, *MSSQL*, *UDP*, *UDP-Lag* e *SYN*) nella giornata dedicata al testing.

Days	Attacks	Attack Time
First Day	PortMap	9:43 - 9:51
	NetBIOS	10:00 - 10:09
	LDAP	10:21 - 10:30
	MSSQL	10:33 - 10:42
	UDP	10:53 - 11:03
	UDP-Lag	11:14 - 11:24
	SYN	11:28 - 17:35
Second Day	NTP	10:35 - 10:45
	DNS	10:52 - 11:05
	LDAP	11:22 - 11:32
	MSSQL	11:36 - 11:45
	NetBIOS	11:50 - 12:00
	SNMP	12:12 - 12:23
	SSDP	12:27 - 12:37
	UDP	12:45 - 13:09
	UDP-Lag	13:11 - 13:15
	WebDDoS	13:18 - 13:29
	SYN	13:29 - 13:34
	TFTP	13:35 - 17:15

Figura 3.3: CIC-DDoS2019: Attacchi Eseguiti

3.1.4 CIC IoT-DIAD

Il dataset *CIC IoT-DIAD* è progettato sia per l'identificazione dei dispositivi *IoT* che per il rilevamento delle anomalie, con lo scopo di far avanzare le applicazioni di analisi della sicurezza negli ambienti *IoT* reali. Per raggiungere tale scopo, sono stati condotti 33 attacchi distinti all'interno di una topologia *IoT* composta da 105 dispositivi presso il *CIC*.

Questi attacchi sono classificati in sette categorie: *DDoS*, *DoS*, *Recon*, *basati su Web*, *Brute Force*, *Spoofing* e *Mirai*. Tutti gli attacchi vengono eseguiti da dispositivi *IoT* malevoli, i quali hanno come obiettivo altri dispositivi *IoT*.

L'approccio proposto sfrutta tecniche di estrazione delle *feature* sia *packet-based* che *flow-based*, in modo da estrarre un insieme di *feature* vario ed essenziale per garantire un rilevamento efficace delle anomalie e un'accurata classificazione dei dispositivi. Questo insieme di *feature* integra un'ampia gamma di attributi provenienti da diversi domini, inclusi elementi relativi ad *HTTPS*, informazioni sull'handshake e stringhe di *User Agent*, estratti nello specifico per l'identificazione dei dispositivi *IoT*. In aggiunta, l'insieme delle funzionalità include attributi specializzati per il rilevamento delle anomalie, come metriche di stream, canale e jitter, calcolate su differenti intervalli temporali per potenziare le capacità di rilevamento delle anomalie del modello.

3.1.5 Risultati Ottenuti

Sono stati addestrati 2 modelli distinti, tramite reti neurali, utilizzando i 2 macro dataset riportati nella figura 3.1, i quali sono stati suddivisi a loro volta in *train*, *test* e *validation*.

Le reti neurali sono tra gli algoritmi più influenti nel *ML* moderno e nell'intelligenza artificiale. L'ispirazione viene dai neuroni biologici del cervello umano, che comunicano tramite segnali elettrici. Una rete neurale è composta principalmente da 3 livelli o strati (*layer*) di neuroni:

- **Livello di input:** Riceve i dati iniziali in ingresso;
- **Strati nascosti:** Sono costituiti da neuroni artificiali che trasformano gli input in nuove rappresentazioni. Matematicamente, i layer nascosti vengono espressi come caratteristiche di input, moltiplicate per i pesi associati e si aggiunge la distorsione per passare da un layer al layer successivo, arrivando infine al layer di output finale. È qui che avviene la trasformazione lineare tra input e output;
- **Livello di output:** Dopo aver eseguito la trasformazione lineare nel livello nascosto, viene aggiunta una funzione di attivazione non lineare (*tanh*, *sigmoid*, *ReLU*) per produrre la previsione finale.

Grazie ad essi la rete impara a trasformare le caratteristiche grezze in modelli significativi e a utilizzarli per fare previsioni.

La tabella seguente mostra i dettagli di ogni modello e come ognuno di essi risponde alla prova effettuata col proprio set di *test*.

Dataset	Output Classes	Parametri Totali	Test Accuracy
CIC-IDS-Collection	8	17768	0.9567
CIC IoT-DIAD	8	17384	0.7381

Figura 3.4: Accuracy dei modelli CIC

I parametri totali sono l'insieme complessivo dei valori numerici (pesi e bias) memorizzati nella rete neurale che determinano il modo in cui il modello elabora gli input per generare gli output. I *pesi* sono i principali elementi di controllo di un modello e determinano come esso valuta nuovi dati e formula previsioni. I *bias* consentono ai modelli di regolare gli output indipendentemente dai pesi e dagli input, fungendo da soglie o offset. I bias aiutano i modelli a generalizzare e catturare modelli e tendenze più ampi in un set di dati.

Le classi di output di un modello di intelligenza artificiale sono le categorie o le etichette finali che il modello può scegliere per classificare un dato in arrivo. Balza subito all'occhio come il dataset *CIC-IDS-Collection* presenti delle prestazioni più elevate in termini di *accuracy*. Mentre l'altro dataset presenta delle prestazioni non ottimali, avendo un *accuracy* bassa, il *CIC-IDS-Collection* dispone di un eccellente risultato avendo una soglia molto ridotta dell'errore per via di un *accuracy* pari a 0.9567.

Lo step successivo è stato quello di utilizzare il modello migliore per analizzare i file *PCAP* riguardanti il traffico dati dell'azienda. Questo test non ha prodotto i risultati sperati, perchè, il traffico dati analizzato dal *CIC* e quello generato all'interno dell'azienda, hanno una struttura differente e quindi il modello non riusciva a classificare il traffico normale e quello anomalo. In poche parole, la configurazione del modello va in *overfit*, ovvero si adatta in modo eccessivo ai dati del *training set*, fallendo nel fare previsioni accurate su dati nuovi o mai visti.

Questa è stata anche una sfida per sperimentare un dataset complesso come quello proposto dal *CIC*, in cui però erano presenti dei dubbi iniziali sulla riuscita della creazione di un modello funzionante addestrato con tale dataset. Per concludere, l'idea riportata all'interno del *paper* da *Janati Idrissi et al.*, è stata scartata e si è pensato di creare un proprio dataset interno all'azienda.

3.2 Log Files

Come introdotto nel paragrafo precedente, si è optato per costruire un dataset adeguato allo scopo del progetto e questo ha richiesto uno studio addizionale per capire quale potesse essere la forma migliore per poter garantire un successo nella riuscita dell'addestramento. Non si ha più un dataset già costruito in un modo strategico come quello del CIC, ma si deve partire da zero fino ad ottenere un risultato efficiente.

La *Survey* intitolato: *Log File Analysis Based on Machine Learning*, scritto da *Rawand Raouf Abdalla, Alaa Khalil Jumaa* e pubblicato nel 2022 presso *UHD JOURNAL OF SCIENCE AND TECHNOLOGY*, evidenzia la crescente importanza dell'analisi dei file di *log* mediante tecniche di *ML* per l'identificazione e la risoluzione di problematiche di sicurezza.

Un file di *log* è un documento digitale che registra in ordine cronologico tutte le attività, gli eventi e le operazioni eseguite da un sistema informatico, un software o un server. Ogni riga di un log rappresenta una specifica voce o evento e include solitamente:

- **Timestamp:** data e ora esatta dell'evento;
- **Evento/Azione:** cosa è successo (es. avvio di un servizio, errore di rete, file modificato);
- **Utente o IP:** chi ha eseguito l'azione o l'indirizzo *IP* del dispositivo richiedente;
- **Stato dell'operazione:** esito positivo o codice di errore.

I principali utilizzi sono:

- **Risoluzione dei problemi:** permettono agli sviluppatori di capire esattamente perché un software o un sistema si è bloccato o presenta un bug;
- **Sicurezza e Audit:** monitorano tentativi di accesso non autorizzati, attacchi informatici o modifiche critiche ai dati;
- **Analisi Web e SEO:** i server web registrano le visite degli utenti e i passaggi dei bot dei motori di ricerca per analizzare le prestazioni del sito;
- **Conformità normativa:** molti regolamenti sulla privacy richiedono la tracciabilità e la conservazione dei log per gli accessi ai dati sensibili.

Il lavoro richiama diversi contributi di rilievo presenti in letteratura. Tra questi viene citato lo studio di Zeufack *et al.*, incentrato su un *framework* di *anomaly detection* non supervisionato — un paradigma di *ML* che non richiede dati etichettati in fase di addestramento — ideato per il tracciamento delle anomalie in tempo reale a partire dai file di *log* di rete.

Un ulteriore contributo esaminato riguarda lo studio pubblicato da *IEEE* nel 2020, intitolato "*Analyzing data granularity levels for insider threat detection using machine learning*". Tale lavoro analizza diverse metodologie di *ML*, tra cui *Neural Networks* (NN), *Random Forest* (RF), *Logistic Regression* (LR) e *XGBoost* (XG), con l'obiettivo di supportare gli analisti di *cybersecurity* nella rilevazione di nuove minacce.

Entrambe le ricerche evidenziano eccellenti prestazioni in termini di accuratezza. Tali evidenze hanno fornito il presupposto metodologico per la scelta del *dataset* di partenza da impiegare nell'addestramento del modello.

L'approccio adottato per il progetto di ricerca *CLEAR*, prevede infatti lo sfruttamento dei file di *log* generati dalle macchine virtuali dell'infrastruttura aziendale, contenenti il tracciamento del traffico dati relativo ai siti web sviluppati da *Net Service*.

3.2.1 Logstash vs Fluentd vs Fluent Bit

Raccolto il file *.log*, si è proceduto ad una fase di *data enrichment*. Nello specifico, ogni singola riga del *log* è stata integrata con metadati aggiuntivi, come, ad esempio, una colonna dedicata al volume di byte inviati al minuto da uno specifico indirizzo *IP*. L'obiettivo è incrementare il più possibile il numero di *feature* per ottenere un *dataset* completo e altamente informativo. Di seguito vengono analizzati 3 strumenti idonei a svolgere questo processo di arricchimento dei dati:

- **Fluentd:** software open source data collector che consente di unificare la raccolta e il consumo dei dati per un uso e una comprensione migliori delle informazioni. *Fluentd* cerca di strutturare i dati in formato *JSON* per quanto possibile: ciò gli permette di unificare ogni aspetto dell'elaborazione dei dati di log — raccolta, filtraggio, buffering e invio in output dei log attraverso molteplici sorgenti e destinazioni. L'elaborazione dei dati risulta molto più semplice con il formato *JSON*, poiché possiede una struttura sufficientemente ben definita per essere accessibile, mantenendo al contempo schemi flessibili. Dispone di un sistema di plugin flessibile che consente alla community di estenderne le funzionalità. Gli oltre 500 plugin messi a disposizione dalla community collegano decine di sorgenti e

destinazioni di dati differenti. Sfruttando questi plugin, è possibile iniziare immediatamente a fare un uso migliore dei propri log. È scritto in una combinazione di linguaggi *C* e *Ruby* e richiede pochissime risorse di sistema. Un'istanza base (*vanilla*) richiede 30-40 MB di memoria ed è in grado di elaborare fino a 13.000 eventi al secondo per singolo core (Events/second/core). Supporta il buffering basato sia su memoria RAM che su disco per prevenire la perdita di dati tra i nodi. Offre inoltre un solido supporto al *failover* e può essere configurato per garantire un'elevata disponibilità.

- **Fluent Bit:** software open source leggero, altamente scalabile e ad altissime prestazioni per la raccolta, l'elaborazione e l'inoltro di log, metriche e tracce (telemetria). Originariamente creato come progetto "satellite" di *Fluentd* per ambienti con risorse molto limitate (come dispositivi *IoT* o *edge*), *Fluent Bit* è diventato lo standard di fatto per il log forwarding negli ambienti containerizzati e *Kubernetes*. Scritto interamente in *C* puro, non ha dipendenze complesse. Utilizza solo 1-2 MB di RAM e pochissima CPU, risultando fino a 10 volte più leggero di *Fluentd*.
- **Logstash:** motore open source di raccolta, elaborazione e centralizzazione dei dati sviluppato da *Elastic*. È il componente chiave di "ingestione" dello stack tecnologico *ELK* (*Elasticsearch*, *Logstash*, *Kibana*). Il suo compito principale è raccogliere log, eventi e metriche da sorgenti eterogenee, trasformarli e pulirli in tempo reale, e inviarli a una o più destinazioni (tipicamente *Elasticsearch* per la successiva analisi su *Kibana*).

La tabella sottostante mostra le differenze fra i 3 software:

Logstash	FluentD	FluentBit
Requires Java runtime environment	Runs on C and Ruby and doesn't need Java dependencies	Designed lightweight.
It has close to 200 plugins, and it is centralized	It has over 500 plug-ins and decentralized. It also requires more memory and CPU. Less need for external plugins	High performance with low-latency data collection and forwarding
No in-built buffering and it uses Redis and Kafka	Built-in buffering, reliable without external dependencies	uses a config file with straight syntax and has basic capabilities for filtering, parsing and buffering
It consumes more memory	memory-efficient	easy to setup with minimal configuration
used for more complex configuration	ideal for centralized logging, complex log aggregation	ideal for edge computing, IoT devices and scenarios for light-weight log forwarding

Figura 3.5: Logstash vs Fluentd vs Fluent Bit

Poiché il contesto operativo richiede l'analisi e la condivisione del traffico dati in tempo reale tra più nodi, requisiti quali la velocità di elaborazione e il ridotto consumo di risorse risultano fondamentali. Fra le tre opzioni esaminate, la scelta è ricaduta su *Fluent Bit*, in quanto garantisce le prestazioni più elevate a fronte di un'occupazione di memoria estremamente contenuta. Inoltre, in relazione al tipo di arricchimento da applicare ai log, *Fluent Bit* soddisfa pienamente i requisiti del progetto senza introdurre la complessità architetturale tipica di strumenti quali *Logstash* o *Fluentd*.

3.2.2 Come funziona Fluent Bit

Fluent Bit possiede un proprio metodo per recuperare, organizzare, modificare e inoltrare tutte le informazioni che gestisce. Questo processo prende il nome di *Data Pipeline* e rappresenta il percorso che tutte le informazioni recuperate dai plugin di input di *Fluent Bit* devono necessariamente seguire.

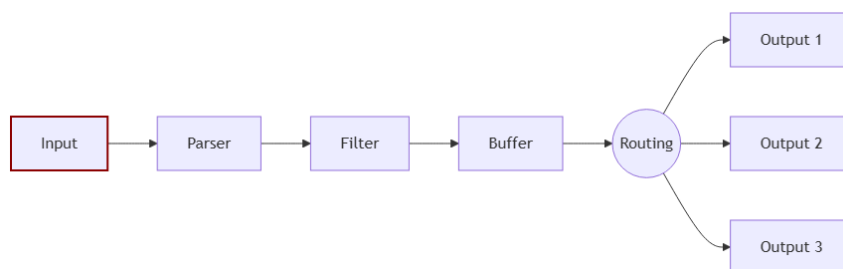


Figura 3.6: Input

L'input rappresenta il primo stadio della *Data Pipeline* e consiste nel recuperare informazioni dalle sorgenti utilizzando le configurazioni dei singoli plugin di input. Questi raccolgono informazioni da fonti differenti. Alcuni plugin acquisiscono dati da file di *log*, mentre altri raccolgono metriche dal sistema operativo e la loro diversità è ampia in modo da consentire la gestione delle esigenze più svariate.

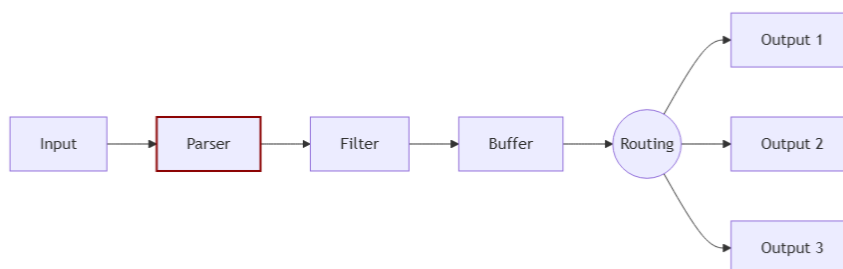


Figura 3.7: Parser

I parser modificano i dati non strutturati acquisiti dai plugin di input, così da semplificare la successiva elaborazione e analisi. Questa modifica avviene prima che *Fluent Bit* applichi qualsiasi filtro o modifichi tali dati. Ciascun plugin di input può avere un solo parser attivo, tuttavia, più plugin all'interno dello stesso file di configurazione di *Fluent Bit* possono utilizzare il medesimo parser oppure impiegarne di differenti.

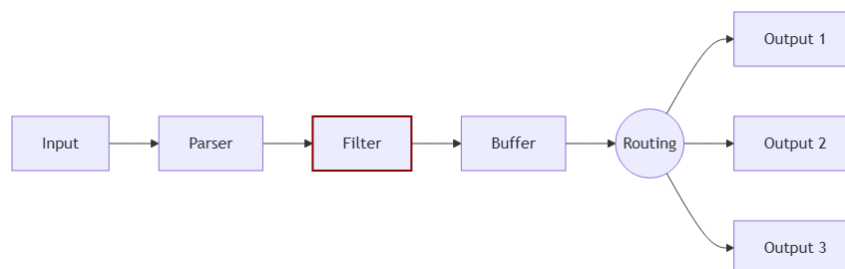


Figura 3.8: Filter

In ambienti di produzione è necessario un controllo completo sui dati raccolti. Il filtraggio consente di alterare i dati acquisiti prima di inviarli ad una destinazione. Questa funzionalità è implementata tramite plugin: ciascun filtro disponibile può essere utilizzato per selezionare (*match*), escludere o arricchire i log con metadati specifici. Analogamente ai plugin di input, i filtri vengono eseguiti nel contesto di un'istanza dedicata, dotata di una configurazione indipendente.

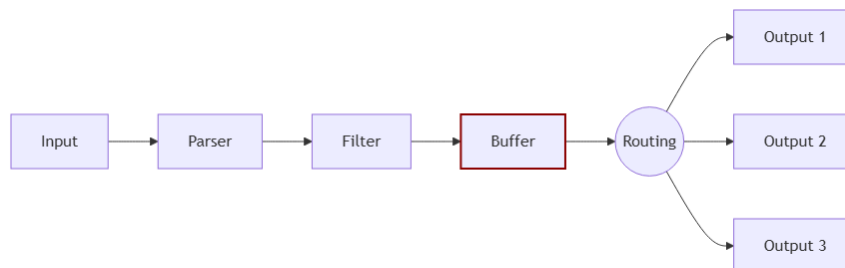


Figura 3.9: Buffer

Dopo che *Fluent Bit* acquisisce i dati, li memorizza temporaneamente nella memoria di sistema (*heap*) prima di elaborarli e instradarli verso la destinazione finale. Questo processo prende il nome di buffering. Quando un plugin di input emette dei record, il motore li raggruppa in blocchi distinti chiamati *chunk* e ognuno di essi ha una dimensione media di 2 MB. La modalità di buffering attiva determina dove questi chunk vengono archiviati.

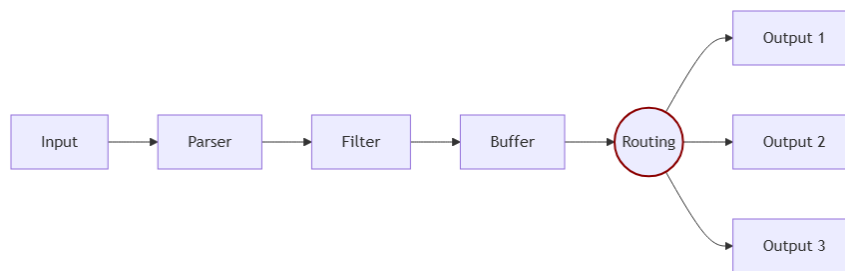


Figura 3.10: Routing

Il routing è una funzionalità chiave che permette di instradare i dati attraverso i filtri fino a una o più destinazioni.

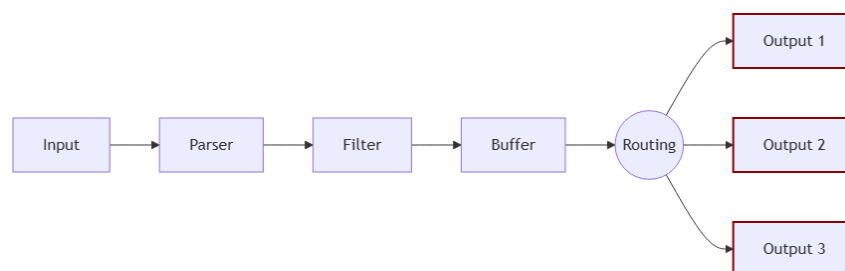


Figura 3.11: Output

Gli Output consentono di definire le destinazioni per i dati. Le più comuni sono i servizi remoti, i filesystem locali o altre interfacce standard. Gli Output sono implementati sotto forma di plugin: quando un plugin di output viene caricato, si crea un'istanza interna ed ognuna di esse possiede una configurazione indipendente.

3.2.3 Log Strutturato

Il punto di partenza dell'analisi consiste nel trasformare il file di *log* aziendale dal suo formato grezzo ad una struttura maggiormente intelligibile e predisposta a successive elaborazioni. Ogni riga del file riporta le informazioni relative alle singole chiamate effettuate da un indirizzo *IP*; tali valori sono separati da uno spazio e privi di una struttura esplicita che ne definisca la semantica, come mostrato di seguito:

```
[14/Jan/2026:07:10:45 +0100] 1.248 www.ordineavvocatibologna.net
77.89.34.226 200 16798 "GET / HTTP/1.1" "-" "Zabbix"
```

Per elaborare questi dati è stata progettata una *Data Pipeline* che acquisisce il *log* di input, vi applica un *parser* e differenti filtri, fino all'ottenimento dell'output finale.

Nello specifico, la fase di input impiega il plugin *tail*, idoneo alla lettura sequenziale di file di testo, e definisce il *parser* da utilizzare.

Quest'ultimo sfrutta il *Regular Expression format (Regex)* per associare una chiave a ciascun elemento informativo. La sintassi *Regex* consente infatti di ricercare, confrontare e manipolare stringhe di testo mediante *pattern* specifici, trasformando il testo grezzo e non strutturato in un formato strutturato.

Attraverso tale operazione, la riga di esempio precedentemente mostrata viene convertita in formato *JSON*, associando a ogni valore la relativa chiave:

```
{"time":"14/Jan/2026:07:10:45 +0100","req_time":"1.248",  
"host":"www.ordineavvocatibologna.net","ip":"77.89.34.226",  
"status":"200","bytes":"16798","method":"GET","path":"/",  
"protocol":"HTTP/1.1","referrer":"-","agent":"Zabbix"}
```

dove:

- **time:** giorno, mese, anno, ore, minuti, secondi e fuso orario corrispondente al tempo dell'Europa Centrale, della richiesta effettuata;
- **req_time:** tempo totale impiegato dal server web per elaborare ed evadere la richiesta, espresso in secondi;
- **host:** nome del dominio richiesto dal client;
- **ip:** indirizzo *IPv4* del client che ha effettuato la richiesta;
- **status:** numero a 3 cifre che identifica l'*HTTP Status Code*;
- **bytes:** volume di dati inviati dal server al client in risposta alla richiesta, espresso in byte;
- **method:** definisce il tipo di azione richiesta dal client, ovvero il metodo *HTTP*;
- **path:** rappresenta la pagina o la risorsa richiesta; "/" indica la root o Home Page del sito web.
- **protocol:** specifica il protocollo di rete utilizzato per il trasferimento della richiesta e della risposta;
- **referrer:** indica l'URL della pagina web da cui l'utente proveniva prima di fare clic sul link verso la pagina attuale. Il valore "-" segnala che il campo è assente;
- **agent:** identifica il software o il client che ha generato la richiesta. In questo caso non si tratta di un browser web come Chrome o Firefox, ma di un agente o web scenario di Zabbix inviato per controllare la disponibilità e le prestazioni del sito.

La fase di filtraggio impiega il plugin *grep*, configurato mediante i parametri *exclude* e *regex*. La prima consente di scartare i record non rilevanti che introdurrebbero ridondanza nel *dataset*, mentre la seconda permette, al contrario, di selezionare ed estrarre unicamente le informazioni d'interesse.

In entrambi i casi, le regole d'inclusione ed esclusione vengono definite tramite *Regex*.

Come plugin di *output* viene impiegato *file*, che istruisce *Fluent Bit* a salvare i dati elaborati all'interno di un file di destinazione. Il percorso ed il nome di tale file vengono definiti rispettivamente mediante i parametri di configurazione *Path* e *File*.

3.2.4 GeoIP

Oltre l'output descritto nella precedente sezione, è stato riportato un'ulteriore sezione per l'output in cui il *log* viene anche salvato all'interno di *Elasticsearch*. *Elasticsearch* è un motore di ricerca e analisi dati distribuito, open source, basato sulla libreria *Apache Lucene*. È sviluppato in *Java* ed è progettato per memorizzare, cercare e analizzare grandi volumi di dati in tempo reale con prestazioni estremamente elevate. A differenza dei database relazionali tradizionali, *Elasticsearch* è un database *NoSQL* orientato ai documenti *JSON* ed è ottimizzato per la ricerca *full-text* e l'aggregazione di log o metriche.

Tramite il plugin *es*, presente all'interno dell'output, *Fluent Bit* è in grado di comunicare con *Elasticsearch* e di inviare i dati precedentemente elaborati dal parser e dai filtri. La configurazione richiede l'indirizzo IP e la porta su cui *Elasticsearch* è in ascolto, mentre l'autenticazione è gestita tramite i parametri *HTTP_User* e *HTTP_Passwd*. Inoltre, l'opzione *tls On* abilita la cifratura del traffico via *HTTPS*, invece la direttiva *tls.verify Off* disabilita la convalida del certificato *SSL/TLS*. Tale configurazione evita ulteriori verifiche a discapito di una vulnerabilità maggiore, ma poco importa perchè siamo in un contesto sperimentale, privo di dati sensibili.

Elasticsearch offre diverse funzionalità per l'analisi dei *log* ricevuti, tra cui la creazione di grafici per il monitoraggio del traffico in tempo reale, oppure consente di eseguire *query* avanzate per filtrare i dati ed estrarre le informazioni di interesse. La visualizzazione e l'interazione con tali dati avvengono tramite *Kibana*, la componente che funge da interfaccia grafica tra l'utente e il motore di ricerca di *Elasticsearch*.

Il lavoro svolto con *Elasticsearch* si è focalizzato principalmente sull'integrazione della componente *GeoIP*, al fine di arricchire i dati con le informazioni relative alla posizione geografica degli indirizzi *IP* che effettuano le richieste al server.

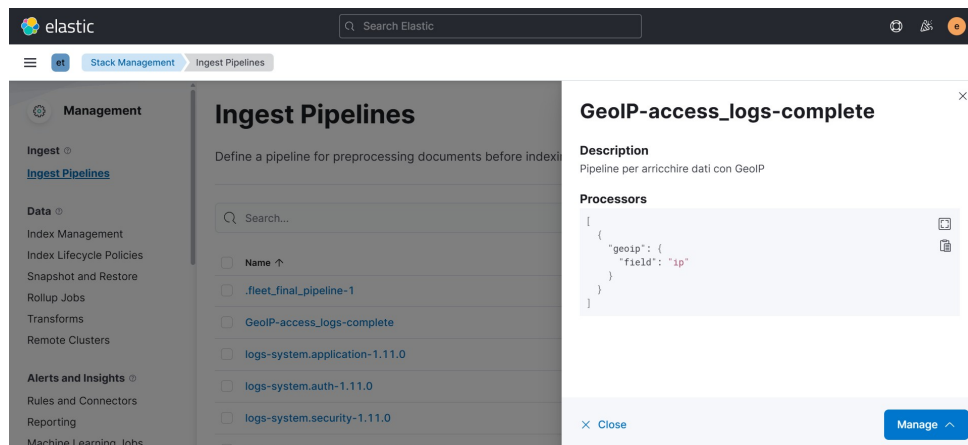


Figura 3.12: Ingest Pipeline di GeoIP

Una *Ingest Pipeline* è una sequenza di operazioni che modifica, arricchisce o trasforma un documento *JSON*. Ogni volta che *Fluent Bit* invia i dati collegandosi a questa pipeline, *Elasticsearch* prende il valore presente nel campo *ip* del documento, individua le coordinate geografiche, il paese e la città corrispondenti, e arricchisce automaticamente il *log* prima di salvarlo.

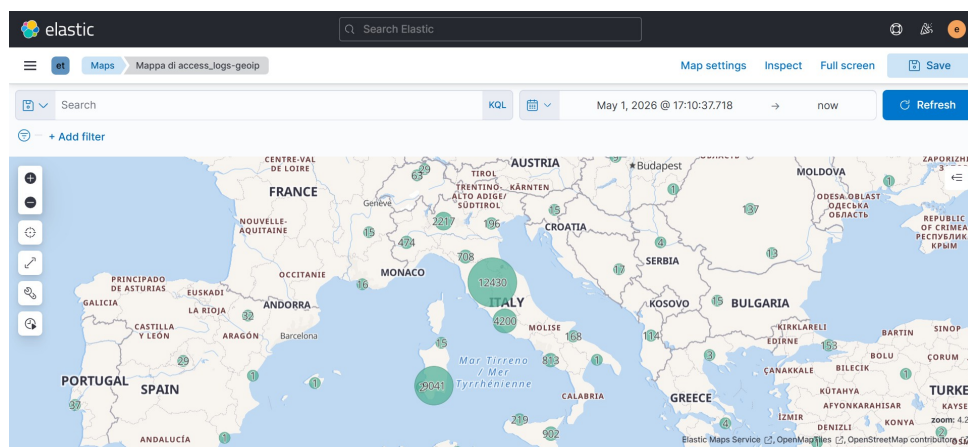


Figura 3.13: Mappa

La mappa illustra graficamente i risultati del processo descritto. Le aree circolari di colore verde riportano al loro interno un valore numerico, il quale indica il totale delle richieste provenienti da quella specifica posizione geografica. Sebbene l'integrazione di *GeoIP* fornisca un ulteriore arricchimento informativo ai dati di *log*, tale componente non verrà momentaneamente impiegata. In questa fase iniziale del lavoro, infatti, l'obiettivo primario è individuare la metodolo-

gia più idonea all'addestramento del modello. L'analisi della geolocalizzazione verrà pertanto integrata in una fase successiva del progetto.

3.2.5 CSV

La fase finale del flusso di lavoro sviluppato con *Fluent Bit* riguarda l'arricchimento informativo dei dati. A tal fine, è stata progettata una seconda *Data Pipeline* che riceve in input il file di *log* precedentemente strutturato e vi applica uno specifico filtro.

Sfruttando l'integrazione nativa del linguaggio *Lua* in *Fluent Bit*, è stato implementato uno *script* capace di elaborare ogni singola voce del *log* (*record*) per generare un file in formato CSV, arricchito con *feature* aggiuntive. Per integrare tale logica, è stato configurato un filtro definendo il parametro *Name* con il valore *lua*, la proprietà *script* con il percorso del file *.lua* e il campo *call* con il nome della funzione da invocare.

Lua è un linguaggio di *scripting* potente, efficiente, leggero ed integrabile. Supporta diversi stili di programmazione: procedurale, orientato agli oggetti, funzionale, data-driven e di descrizione dei dati. Vanta una meritata reputazione in termini di prestazioni. Diversi benchmark dimostrano che *Lua* è il linguaggio più veloce nell'ambito dei linguaggi di *scripting* interpretati. Offre un'API semplice e ben documentata che permette una forte integrazione con il codice scritto in altri linguaggi. È facile estendere *Lua* con librerie scritte in altri linguaggi, così com'è facile estendere programmi scritti in altri linguaggi utilizzando *Lua*.

All'atto dell'invocazione, la funzione estrae e processa unicamente i campi rilevanti, quali `record["host"]`, `record["ip"]`, `record["method"]`, `record["status"]`, `record["data"]` e `record["bytes"]`. La notazione `record["feature"]` individua la specifica variabile analizzata all'interno del *record* corrente.

La strategia adottata prevede il raggruppamento delle richieste client che condividono il medesimo indirizzo *IP*, lo stesso *host*, il medesimo metodo *HTTP*, lo stesso *status code* e una medesima finestra temporale, impostata su un intervallo di 30 secondi. Nel caso in cui anche un solo parametro differisca, le chiamate vengono trascritte su righe distinte. L'intervallo temporale viene tracciato registrando il *timestamp* della prima e dell'ultima richiesta pervenuta nella finestra di osservazione (i due valori coincidono qualora venga eseguita una sola chiamata).

Rispetto al formato iniziale, questa aggregazione integra due nuove informazioni per ciascun record: il numero totale di chiamate eseguite e il volume complessivo di byte inviati nell'arco dei 30 secondi.

```

1775835491.000000000,"20x","78.213.25.67","10/Apr/2026:15:38:06-10/Apr/2026:15:38:06",1,22356,"GET","www.sardegna-solidale.it"
1775835491.000000000,"30x","151.59.134.10","10/Apr/2026:15:37:55-10/Apr/2026:15:38:02",14,70,"GET","www.sardegna-solidale.it"
1775835491.000000000,"20x","151.59.134.10","10/Apr/2026:15:37:54-10/Apr/2026:15:38:01",7,19040,"POST","www.sardegna-solidale.it"
1775835491.000000000,"20x","80.211.146.125","10/Apr/2026:15:37:56-10/Apr/2026:15:37:56",1,31,"POST","www.sardegna-solidale.it"
1775835523.000000000,"20x","116.179.32.19","10/Apr/2026:15:38:11-10/Apr/2026:15:38:11",1,15617,"GET","www.sardegna-solidale.it"
1775835523.000000000,"20x","72.14.201.23","10/Apr/2026:15:38:40-10/Apr/2026:15:38:40",1,18442,"GET","www.sardegna-solidale.it"
1775835523.000000000,"20x","151.59.134.10","10/Apr/2026:15:38:23-10/Apr/2026:15:38:28",10,26070,"GET","www.sardegna-solidale.it"
1775835523.000000000,"20x","136.243.220.209","10/Apr/2026:15:38:17-10/Apr/2026:15:38:17",1,15621,"GET","www.sardegna-solidale.it"
1775835523.000000000,"30x","151.59.134.10","10/Apr/2026:15:38:23-10/Apr/2026:15:38:27",10,50,"GET","www.sardegna-solidale.it"
1775835523.000000000,"20x","151.59.134.10","10/Apr/2026:15:38:22-10/Apr/2026:15:38:27",5,13600,"POST","www.sardegna-solidale.it"
1775835523.000000000,"20x","151.59.134.10","10/Apr/2026:15:38:44-10/Apr/2026:15:38:49",10,26070,"GET","www.sardegna-solidale.it"
1775835523.000000000,"20x","125.167.58.147","10/Apr/2026:15:38:47-10/Apr/2026:15:38:47",1,16712,"GET","www.sardegna-solidale.it"
1775835553.000000000,"30x","151.59.134.10","10/Apr/2026:15:38:44-10/Apr/2026:15:38:49",10,50,"GET","www.sardegna-solidale.it"
1775835553.000000000,"20x","151.59.134.10","10/Apr/2026:15:38:43-10/Apr/2026:15:38:48",5,13600,"POST","www.sardegna-solidale.it"
1775835594.000000000,"20x","151.59.134.10","10/Apr/2026:15:39:19-10/Apr/2026:15:39:36",30,78210,"GET","www.sardegna-solidale.it"
1775835594.000000000,"20x","34.19.37.246","10/Apr/2026:15:39:24-10/Apr/2026:15:39:24",1,138676,"GET","www.sardegna-solidale.it"
1775835594.000000000,"20x","185.191.171.17","10/Apr/2026:15:39:14-10/Apr/2026:15:39:14",1,14951,"GET","www.sardegna-solidale.it"
1775835594.000000000,"30x","151.59.134.10","10/Apr/2026:15:39:19-10/Apr/2026:15:39:35",30,150,"GET","www.sardegna-solidale.it"
1775835594.000000000,"20x","80.211.146.125","10/Apr/2026:15:39:13-10/Apr/2026:15:39:13",1,31,"POST","www.sardegna-solidale.it"
1775835594.000000000,"20x","151.59.134.10","10/Apr/2026:15:39:18-10/Apr/2026:15:39:35",15,40800,"POST","www.sardegna-solidale.it"
1775835629.000000000,"20x","151.59.134.10","10/Apr/2026:15:40:01-10/Apr/2026:15:40:06",10,26070,"GET","www.sardegna-solidale.it"
1775835629.000000000,"20x","136.243.220.209","10/Apr/2026:15:39:54-10/Apr/2026:15:39:54",1,14659,"GET","www.sardegna-solidale.it"
1775835629.000000000,"30x","151.59.134.10","10/Apr/2026:15:40:01-10/Apr/2026:15:40:05",10,50,"GET","www.sardegna-solidale.it"
1775835629.000000000,"20x","80.211.146.125","10/Apr/2026:15:40:00-10/Apr/2026:15:40:00",1,31,"POST","www.sardegna-solidale.it"
1775835629.000000000,"20x","151.59.134.10","10/Apr/2026:15:40:00-10/Apr/2026:15:40:05",5,13600,"POST","www.sardegna-solidale.it"
1775835641.000000000,"40x","87.20.80.97","10/Apr/2026:15:40:40-10/Apr/2026:15:40:40",2,324,"GET","www.sardegna-solidale.it"
1775835641.000000000,"20x","151.59.134.10","10/Apr/2026:15:40:30-10/Apr/2026:15:40:40",15,39105,"GET","www.sardegna-solidale.it"
1775835641.000000000,"20x","51.195.183.22","10/Apr/2026:15:40:29-10/Apr/2026:15:40:29",1,14762,"GET","www.sardegna-solidale.it"
1775835641.000000000,"20x","87.20.80.97","10/Apr/2026:15:40:40-10/Apr/2026:15:40:40",3,251163,"GET","www.sardegna-solidale.it"
1775835641.000000000,"30x","151.59.134.10","10/Apr/2026:15:40:30-10/Apr/2026:15:40:39",15,75,"GET","www.sardegna-solidale.it"
1775835641.000000000,"20x","151.59.134.10","10/Apr/2026:15:40:29-10/Apr/2026:15:40:40",10,27200,"POST","www.sardegna-solidale.it"

```

Figura 3.14: CSV

L'estratto riportato in figura illustra la struttura del file CSV così ottenuto, il quale costituirà il dataset per il successivo addestramento del modello.

Scorrendo le righe da sinistra verso destra abbiamo:

- **timestamp:** *Unix Timestamp* espresso in secondi e nanosecondi. La parte intera rappresenta i secondi trascorsi dall'inizio dell'*Epoca Unix* (ossia dall'1 gennaio 1970 alle 00:00:00 UTC), mentre la parte decimale rappresenta le frazioni di secondo con precisione fino ai nanosecondi;
- **status code:** numero che specifica l'esito della richiesta in cui l'ultima cifra è irrilevante e per questo motivo viene segnata solamente una *x*;
- **IP:** stringa numerica che indica l'indirizzo *IP*;
- **data:** intervallo temporale di 30 secondi in cui viene espressa la data di inizio e fine in un formato più congeniale alla lettura, in cui vengono espressi: giorno, mese, anno, ora, minuti e secondi;
- **request:** totale delle chiamate effettuate dal *client*;
- **bytes:** volume totale dei bytes inviati dal *client*;
- **method:** azione specifica che un *client* vuole eseguire;
- **host:** server invocato dalle richieste del *client*.

3.2.6 Brute Force

Come si osserva dalla Figura 3.14, l'*host* preso in esame si riferisce costantemente al sito web *Sardegna Solidale*. Inizialmente, infatti, i *log* forniti da *Net Service* sono stati utilizzati come base di studio per comprendere il formato dei dati e adattare la *Data Pipeline* di *Fluent Bit*, nell'ottica di estendere il lavoro senza ulteriori modifiche a un dataset interamente basato sui *log* di *Net Service*. Una volta definita la struttura dei dati, l'attenzione si è focalizzata sui *log* del server *NGINX* relativi al traffico di *Sardegna Solidale*, i quali sono stati opportunamente formattati per renderli omogenei a quelli di *Net Service*.

Quest'ultimo è un sito dell'azienda *Flosslab* programmato tramite *WordPress*, il quale è un *Content Management System*, ovvero un programma gratuito e open source che permette di creare e gestire siti internet, blog o e-commerce senza bisogno di conoscere i linguaggi di programmazione. È irrilevante sapere com'è fatto e cosa riguarda il sito ai fini del progetto.

NGINX è un software open source ad alte prestazioni utilizzato come server web, reverse proxy, load balancer e cache *HTTP*. Utilizza un'architettura *event-driven* e asincrona, permettendogli di gestire decine di migliaia di connessioni simultanee occupando pochissima memoria RAM e risorsa CPU.

La scelta di passare ai *log* di *Sardegna Solidale* deriva dalla collaborazione con il team di *Flosslab*, che gestisce la piattaforma. Avendo accesso diretto al sito web, i ricercatori di *Flosslab* hanno potuto effettuare simulazioni d'attacco controllate, consentendo di monitorare nei file di *log* la presenza di diversi indirizzi *IP* malevoli riconducibili ad attacchi di tipo *Brute Force*.

La simulazione dell'attacco è stata implementata tramite uno *script Python*. Inizialmente, la procedura legge da un file la lista degli *username* e da un altro la lista delle *password*, applicando una fase di *cleaning* per rimuovere spazi bianchi e caratteri di a capo non visibili.

Successivamente, viene invocata una funzione che seleziona casualmente uno *User-Agent* da un elenco predefinito, rappresentativo di diversi *browser* e dispositivi, al fine di eludere eventuali regole di blocco basate sulla firma del client. Viene quindi verificata la presenza di *cookie* in *cache* ed eventualmente la loro validità temporale (se generati da meno di 10 minuti).

L'utilizzo dei *cookie* è fondamentale per 3 motivi:

- **Validazione del test dei cookie di WordPress:** quando un utente visita la pagina *wp-login.php*, WordPress invia un *cookie* temporaneo chiamato *wordpress_test_cookie*. Quando si invia il modulo di login, il server invia tra i dati del modulo il parametro "*testcookie*": "1" e verifica che il browser restituisca contemporaneamente il *cookie* precedentemente assegnato. Se lo script non memorizzasse e restituisse quel *cookie*, WordPress rifiuterebbe il tentativo di login a prescindere dalla correttezza di *username* e *password*;
- **Mantenimento dello stato tra le richieste:** i *cookie* generati dalla richiesta di login (i *cookie* di sessione di WordPress) devono essere memorizzati e inviati con la richiesta *GET* successiva per dimostrare al server che la richiesta proviene dall'utente appena autenticato;
- **Ottimizzazione e riduzione delle richieste:** il codice salva i *cookie* iniziali di navigazione per 10 minuti. Questo evita di dover scaricare da capo la pagina di login a ogni singola combinazione di password provata, riducendo il traffico generato e rendendo i tentativi leggermente più veloci.

Una volta ottenuti i *cookie*, lo script compone e invia i parametri di autenticazione tramite richiesta *POST* alla pagina di login. Contestualmente, viene inoltrata una richiesta *GET* all'area riservata (*wp-admin/*). Se l'URL di destinazione finale a seguito del reindirizzamento corrisponde ancora a *wp-login.php*, il tentativo viene registrato come fallito; in caso contrario, l'autenticazione si considera completata con successo.

Il ciclo continua iterando su tutte le *password* per il primo utente. Terminato il file delle *password*, passa all'utente successivo nel file degli *username* e ripete l'intero processo fino a trovare le credenziali valide o fino all'esaurimento dei file.

Capitolo 4

Training

Il presente capitolo propone uno studio comparativo su quattro differenti algoritmi di intelligenza artificiale, valutati al fine di individuare l'architettura più prestazionale per l'addestramento del modello.

4.1 Data Preprocessing

Prima di descrivere la fase di addestramento, si rende tuttavia necessaria un'introduzione al processo di *Data Preprocessing*.

Questo è il processo di pulizia, trasformazione e organizzazione dei dati grezzi in un formato strutturato adatto all'analisi, al *ML* e alle applicazioni di *IA*. Aiuta a rendere i dati più accurati, coerenti e idonei alla modellazione rimuovendo il rumore, gestendo i valori mancanti, rimuovendo i duplicati e standardizzando le *feature*. Un *Preprocessing* efficace dei dati:

- **Migliora la qualità dei dati:** elimina incoerenze, ridondanze ed errori;
- **Migliora le prestazioni del modello:** favorisce una maggiore accuratezza ed efficienza negli algoritmi di *ML*;
- **Riduce la complessità computazionale:** ottimizza le strutture dati per un'elaborazione più rapida;
- **Crea intuizioni migliori:** porta a una *business intelligence* e a un processo decisionale più affidabile.

Infine si arriva alla parte di *Data Splitting*, in cui avviene una suddivisione dei dati in set di *training*, *validation* e *test* per la valutazione del modello di *ML*.

Nello specifico il *training set* include i dati che verranno utilizzati esclusivamente in fase di addestramento del modello. Vuol dire che il nostro modello imparerà le relazioni tra le nostre le variabili di input (X) e quelle di output (Y), ovvero le variabili binarie che vogliamo predire. Il modello così confronta il risultato della sua predizione con quello reale e di conseguenza aggiorna i vari parametri per ridurre al minimo l'errore rispetto alla volta precedente.

Per evitare l'*overfitting*, per avere quindi una reale capacità predittiva, diamo al nostro modello dei dati che non ha mai visto e gli diciamo di fare una previsione. Questi dati devono essere esattamente come quelli di *train* e questi dati fanno parte del *validation set*. A questo punto avremo una *Y* predetta, e una *Y* reale e ci basterà confrontarle per vedere quanto il nostro modello riesce a prevedere con una buona approssimazione la nostra variabile di output. Si chiama *validation set* perché si occupa di validare i risultati ottenuti nel *training set*. Se le performances fossero scarse, dovremmo andare a modificare gli iperparametri del modello e ripartire con il *train*, finché il risultato sul *validation* non ci soddisferà.

Infine, una volta che il nostro modello riuscirà ad avere delle buone *performances* sul *train set* e soprattutto sul *validation set*, possiamo testare il nostro modello su altre osservazioni che il modello non ha mai visto, osservazioni che fanno parte del *test set*. Questo è un set che a volte viene anche omesso e serve quasi esclusivamente per valutare e visualizzare le prestazioni e il funzionamento del modello.

I dati più corposi solitamente sono quelli del *train*, perché l'algoritmo ha bisogno di imparare. Subito dopo arriva il *validation*: un *validation* troppo grande rischia di sottrarre dati all'apprendimento, ma uno troppo piccolo rischia poi di inserire un *bias* troppo forte in sede di valutazione e quindi di invalidare le capacità predittive del modello. Infine, una minima parte viene dedicata al *test set*. Solitamente, una buona divisione è quella di utilizzare 70% dei dati per il *train*, il 20% per il *validation* e il 10% per il *test*.

Mediante la libreria *Python pandas*, è stato acquisito il file CSV contenente i log analizzati nel capitolo precedente. Nel dataset è stata introdotta una colonna denominata *label*, alla quale è associato il valore "1" per le righe relative a indirizzi IP malevoli (ovvero quelli coinvolti nell'attacco di tipo *Brute Force*) e "0" negli altri casi.

Successivamente, è stata calcolata la frequenza dei singoli indirizzi IP, integrata come nuova *feature*. Tale metrica esprime il numero totale di occorrenze di ciascun IP all'interno del file di log, rappresentando il volume complessivo di richieste effettuate.

Per rendere i dati elaborabili dai modelli di *ML*, che richiedono variabili numeriche, la colonna categorica *Method* è stata rimossa e sostituita mediante la tecnica di *One-Hot Encoding* con tre variabili binarie: *Method_GET*, *Method_POST* e *Method_OTHER*. Nello specifico, per ciascuna richiesta viene assegnato il valore "1" alla colonna corrispondente al metodo *HTTP* utilizzato e "0" alle rimanenti. Sono state inoltre rimosse diverse colonne prive di potere informativo ai fini dell'addestramento:

- **Host:** rimosso in quanto costante per tutti i record del *dataset*;
- **Intervallo Temporale:** eliminato poiché il formato della stringa temporale originale (*Giorno/Mese/Anno:Ora:Minuti:Secondi-Giorno/Mese/Anno:Ora:Minuti:Secondi*) non è direttamente interpretabile dal modello. La variabile è stata sostituita dalla colonna *duration_sec*, che calcola la durata in secondi tra la prima e l'ultima richiesta della finestra di osservazione (pari a 30 secondi);
- **IP:** sostituiti dalla rispettiva frequenza assoluta;
- **Status Code:** omesso in quanto l'applicazione *WordPress* restituisce il codice "200 OK" anche in corrispondenza di pagine d'errore (come il "404 Not Found").

Il *dataset* è stato ripulito mediante la rimozione dei record contenenti valori mancanti (*N/A*) o valori negativi non coerenti.

Sulle *feature* continue — nello specifico *duration_sec*, *Response_bytes* (volume di byte trasmessi nei 30 secondi), *IP_freq* e *Total* (numero totale di richieste nei 30 secondi) — è stata applicata una trasformazione logaritmica nella forma $\log(1+x)$. Tale operazione consente di ridurre la componente di asimmetria (*skewness*) generata dai picchi estremi tipici del traffico di rete, rendendo la distribuzione dei dati maggiormente assimilabile a una curva normale. Infine, le variabili numeriche sono state normalizzate nell'intervallo $[0, 1]$ tramite la tecnica di *MinMaxScaler*. Questo aiuta i modelli di *ML* ad addestrarsi in modo più efficace, garantendo che tutte le caratteristiche contribuiscano equamente al modello scalandole nello stesso intervallo. Inoltre questo aiuta a ridurre la scala dei dati all'intervallo richiesto e ne migliora la visualizzazione. La formula matematica è la seguente:

$$x_{scaled} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Di seguito si riporta un estratto della struttura dati risultante dalle fasi di pre-elaborazione descritte:

	duration_sec	Total	Response_bytes	IP_freq	Method_GET	Method_OTHER	Method_POST	label
0	0.0	0.0	0.668448	0.000000	1	0	0	0
1	0.0	0.0	0.549385	0.118007	1	0	0	0
2	0.0	0.0	0.197088	0.961558	0	0	1	0
3	0.0	0.0	0.347544	0.000000	1	0	0	0
4	0.0	0.0	0.547924	0.000000	1	0	0	0

Figura 4.1: Data Preprocessing applicato al CSV

4.2 Random Forest

A partire dall'insieme di *feature* individuate, è stata condotta un'ulteriore analisi volta a selezionare le variabili maggiormente informative, al fine di ridurre la complessità computazionale e alleggerire il processo di addestramento. Tale selezione si basa sull'impiego dell'algoritmo *Random Forest*, ampiamente documentato in letteratura per la valutazione e l'estrazione dell'importanza delle variabili (*feature importance*).

Il *random forest* è un algoritmo di apprendimento automatico supervisionato, registrato da *Leo Breiman* e *Adele Cutler*, che combina l'output di più *decision trees* per raggiungere un unico risultato. La sua facilità d'uso e la sua flessibilità ne hanno favorito l'adozione, in quanto gestisce sia i problemi di classificazione che quelli di regressione.

Un *decision tree* è un algoritmo di apprendimento supervisionato non parametrico, il quale presenta una struttura gerarchica ad albero che consiste in un nodo radice, rami, nodi interni e nodi foglia.

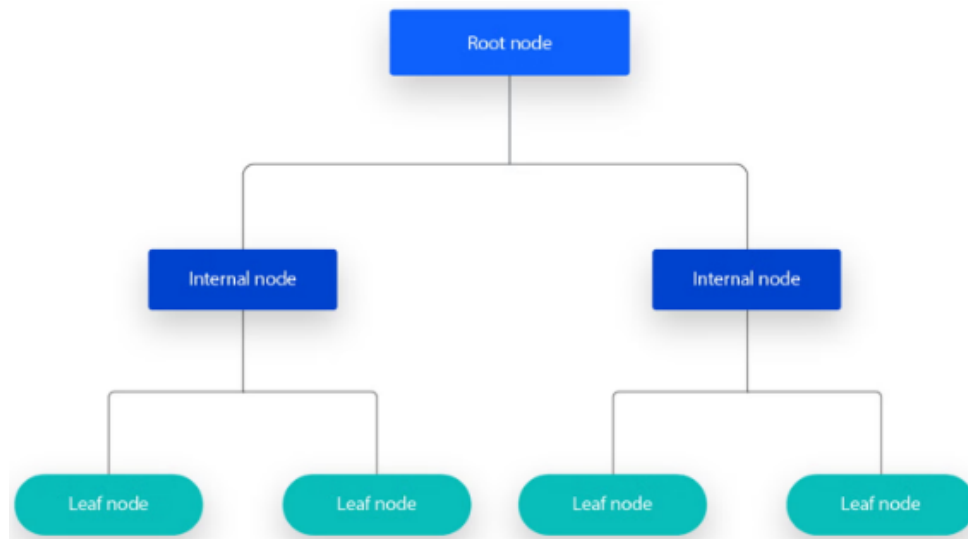


Figura 4.2: Decision Tree

Un *decision tree* inizia con un nodo radice, che non ha rami in entrata. I rami in uscita dal nodo radice alimentano quindi i nodi interni, noti anche come nodi decisionali. Sulla base delle funzionalità disponibili, entrambi i tipi di nodo conducono valutazioni per formare sottoinsiemi omogenei, indicati da nodi foglia o nodi terminali. I nodi foglia rappresentano tutti i possibili risultati all'interno del set di dati. L'apprendimento basato sui *decision tree* utilizza una strategia "*divide et impera*" conducendo una ricerca approfondita per identificare i punti di divisione ottimali all'interno di un albero. Questa suddivisione viene poi ripetuta in modo ricorsivo, dall'alto verso il basso, fino a quando tutti o la maggior parte dei record vengono classificati con etichette di classe specifiche.

Mentre i *decision tree* considerano tutte le possibili suddivisioni delle caratteristiche, il *random forest* seleziona solo un sottoinsieme di tali caratteristiche. Quest'ultimo è costituito da una raccolta di *decision trees* e ogni albero dell'insieme è composto da un campione di dati estratto casualmente da un set di addestramento con sostituzione, chiamato *Bootstrapping*. Di quel campione di allenamento, un terzo viene accantonato come dati di test, noti come campione *out-of-bag (oob)*. Un altro esempio di casualità viene poi immesso attraverso il *feature bagging*, il quale genera un sottoinsieme casuale di caratteristiche aggiungendo maggiore diversità al set di dati e riducendo la correlazione tra *decision trees*. A seconda del tipo di problema, la determinazione della previsione varierà. Per un'attività di regressione, verrà calcolata la media dei singoli *decision trees* e per un'attività di classificazione, un voto di maggioranza (ovvero la variabile categorica più frequente) produrrà la classe prevista.

Infine, il campione *oob* viene quindi utilizzato per la convalida incrociata, finalizzando tale previsione.

Nel presente caso di studio, a valle della fase di *Data Preprocessing*, il *dataset* è stato suddiviso nella matrice delle variabili predittive (privata della colonna *label*) e nel vettore della variabile target (*label*). Entrambe le componenti sono state ulteriormente ripartite in un *train set* (pari all'80% dei dati) e in un *test set* (pari al restante 20%). Successivamente, è stato istanziato il classificatore *RandomForestClassifier* fornito dal modulo *scikit-learn*. Durante l'addestramento, ciascun *decision tree* analizza un sottoinsieme casuale di campioni e di variabili, identificando le soglie di scissione ottimali. Le predizioni generate dai singoli alberi vengono aggregate per calcolare la media della probabilità predittiva complessiva, quantificando così il contributo individuale di ogni *feature*. Al termine della procedura, lo *script* estrae i punteggi di importanza normalizzati associati a ciascuna colonna (la cui somma è pari a 1.0).

	Feature	Importance
3	IP_freq	0.447603
1	Total	0.282653
0	duration_sec	0.167705
2	Response_bytes	0.085585
6	Method_POST	0.008329
4	Method_GET	0.008008
5	Method_OTHER	0.000117

Figura 4.3: Punteggi delle *Features*

Come illustrato nella tabella sottostante, la variabile *IP_freq* risulta essere la più rilevante ai fini della classificazione. Al contrario, le variabili relative ai metodi *HTTP* forniscono un apporto marginale; per tale ragione, le colonne *Method_POST*, *Method_GET* e *Method_OTHER* sono state rimosse dal *dataset* finale.

4.3 Isolation Forest

Lo studio condotto da *Wilson Chua et al.* intitolato: *Web Traffic Anomaly Detection Using Isolation Forest* e pubblicato nel 2024 dalla casa editrice *MDPI (Multidisciplinary Digital Publishing Institute)* sulla rivista scientifica *Informatics*, dimostra che *Isolation Forest* costituisce una soluzione rapida ed efficace per l'individuazione di anomalie nel traffico web.

Il modello, implementato in *Python* con *Scikit-learn*, ha registrato prestazioni eccellenti:

- **Accuracy:** 93%. Rappresenta la proporzione di tutte le classificazioni corrette, positive o negative ed è definita matematicamente come:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

dove *TP* e *TN* stanno corrispettivamente per *True Positive* e *True Negative* ed identificano le risposte che il modello asserisce quando effettua una predizione corretta, mentre *FP* ed *FN* vogliono dire *False Positive* e *False Negative* e corrispondono a predizioni errate del modello, rispettivamente al caso in cui il modello riconosce erroneamente un'anomalia e viceversa;

- **Precision:** 95%. Rappresenta la proporzione di tutte le classificazioni positive del modello che sono effettivamente positive. È definita matematicamente come:

$$Precision = TP / (TP + FP)$$

- **Recall:** 90%. Rappresenta la proporzione di tutti i positivi effettivi che sono stati classificati correttamente come positivi.

$$Recall = TP / (TP + FN)$$

- **F1-score:** 92%. Risolve la tensione tra *precision* e *recall* combinandoli in un unico numero tramite la media armonica

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Isolation Forest è un algoritmo utile ed efficiente impiegato per il rilevamento delle anomalie, caratteristica che lo rende una scelta molto diffusa in settori come la sicurezza informatica, la finanza e la sanità. Funziona isolando i punti dati che differiscono in modo significativo dalle normali osservazioni attraverso un partizionamento casuale, cioè i dati vengono suddivisi utilizzando *feature* e valori di soglia selezionati in modo casuale. Poiché le anomalie sono poche e ben distinte, vengono isolate più velocemente rispetto ai dati normali, consentendo una rapida identificazione degli *outlier* con un minimo sforzo computazionale. Una componente chiave è l'*Anomaly Score*, che rappresenta la facilità con cui un punto dati può essere isolato. Un minor numero di divisioni si traduce in un punteggio di anomalia più elevato e, di conseguenza, in una maggiore probabilità che si tratti di un *outlier*.

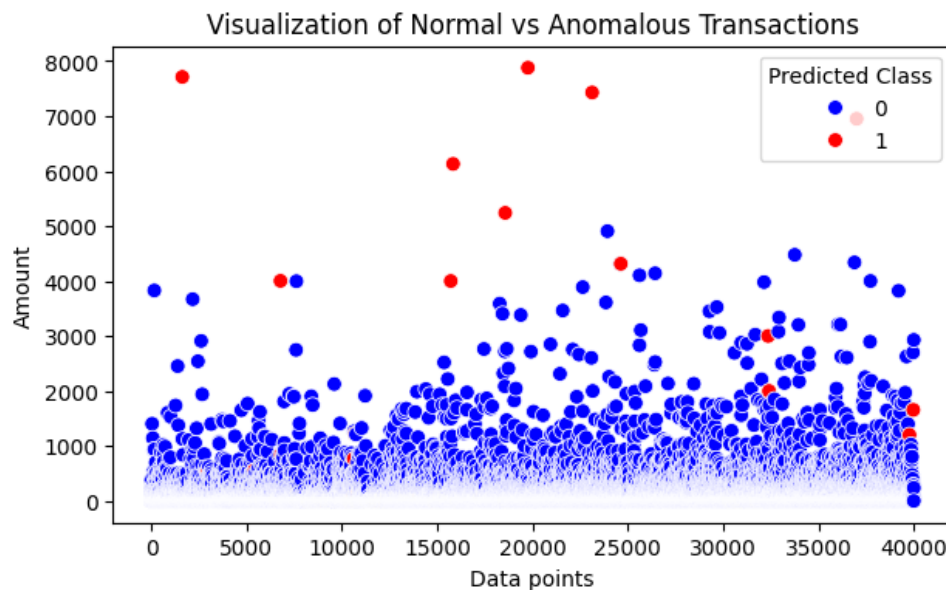


Figura 4.4: Isolation Forest

Come illustrato nel grafico sovrastante, l'algoritmo isola nettamente le istanze anomale (rappresentate dai punti in rosso), le quali risultano numericamente esigue e ben distinte dal profilo di traffico ordinario (rappresentato dai punti in blu).

Nell'ambito del progetto *CLEAR*, l'algoritmo è stato implementato tramite il modulo `IsolationForest` della libreria *scikit-learn* (`sklearn.ensemble`). Poiché l'efficacia di `IsolationForest` dipende criticamente dalla configurazione dei propri parametri, è stata definita una griglia di ottimizzazione degli iperparametri comprendente i seguenti valori:

- **n_estimators:** numero di alberi nella foresta;
- **max_samples:** quantità di dati usati per addestrare ciascun albero;
- **contamination:** percentuale stimata di anomalie nel dataset;
- **max_features:** indicatore di *feature* da selezionare casualmente per ogni albero;
- **bootstrap:** variabile *booleana* che decide se estrarre i dati con o senza reinserimento durante la creazione degli alberi.

Attraverso tale procedura, il modello viene addestrato su ciascuna combinazione di iperparametri specificata, consentendo di selezionare e salvare la configurazione che massimizza le prestazioni in termini di *F1-Score*.

4.4 HDBSCAN

L'articolo di ricerca: *Statistical and density-based clustering techniques in the context of anomaly detection in network systems: A comparative analysis*, scritto da Aleksandr S. Baklashov e Dmitry S. Kulyabov pubblicato nel 2025 dalla rivista scientifica *Discrete & Continuous Models & Applied Computational Science*, confronta diversi algoritmi di clustering per rilevare anomalie e attacchi informatici nel traffico di rete. Gli algoritmi presi in esame sono: *K-means*, *K-medoids*, *Hierarchical Clustering*, *DBSCAN*, *HDBSCAN* e *OPTICS*. L'analisi, condotta sul dataset *NSL-KDD*, ha evidenziato come l'algoritmo *HDBSCAN* offra le prestazioni migliori, presentando i seguenti risultati: *Precision* pari a 0,868%, *Recall* pari a 0,988% ed *F1-score* pari a 0,924%.

Per tale ragione, si è scelto di integrarne l'utilizzo all'interno del progetto *CLEAR*. Per spiegarlo al meglio è necessario fare una breve introduzione al suo predecessore, ovvero *DBSCAN*. L'algoritmo *DBSCAN* (*Density-Based Spatial Clustering of Applications with Noise*) individua i cluster identificando aree ad elevata densità di punti. Il processo inizia selezionando un punto casuale e definendo il suo intorno. Se il numero di punti vicini supera una soglia predefinita *MinPts* (*minimum number of neighbors*), il punto diventa un *core point*. L'algoritmo forma quindi i cluster unendo i *core points* ed i punti di confine (*boundary points*) ad essi correlati. I punti che non appartengono a nessun cluster e non fanno parte di aree dense vengono considerati come *outlier* o rumore (*noise*).

L'*HDBSCAN* (*Hierarchical Density-Based Spatial Clustering of Applications with Noise*) estende il *DBSCAN* incorporando un approccio di clustering gerarchico. Dopo aver eseguito la procedura principale di *DBSCAN*, *HDBSCAN* unisce iterativamente i cluster in base alle distanze che li separano, formando una gerarchia di aree connesse per densità. Nel complesso, quest'ultimo offre una potente tecnica di clustering con diversi vantaggi rispetto al classico *DBSCAN*. Tra questi figurano la determinazione automatica del numero di cluster e una maggiore robustezza nella selezione dei parametri.

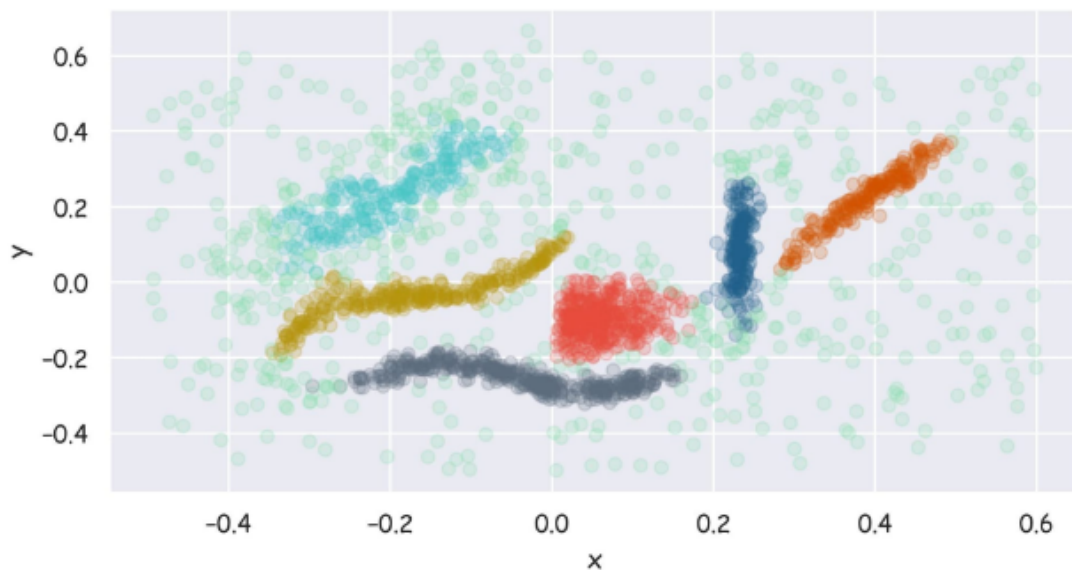


Figura 4.5: HDBSCAN Clustering

Come per l'*Isolation Forest*, è stata definita una griglia di ottimizzazione degli iperparametri:

- **min_cluster_size:** Dimensione minima che un gruppo di punti deve avere per essere considerato un cluster. Raggruppamenti più piccoli di questa dimensione verranno considerati come *noise*;
- **min_samples:** Parametro k utilizzato per calcolare la distanza tra un punto ed il suo k -esimo vicino più adiacente. Più è alto, più punti verranno classificati come rumore/anomalie. Se *None*, prende lo stesso valore di *min_cluster_size*;
- **cluster_selection_epsilon:** Soglia per la quale i cluster al di sotto di questo valore verranno uniti;
- **cluster_selection_method:** Metodo usato per selezionare i cluster dall'albero. L'approccio standard prevede l'utilizzo dell'algoritmo *eom* per trovare il cluster più persistente. Alternativamente, tramite *leaf*, è possibile selezionare i cluster in base alle foglie dell'albero;
- **metric:** Distanza usata per misurare la vicinanza tra i punti;
- **alpha:** Controlla il livello di conservatività del clustering durante la costruzione dell'albero delle distanze;
- **allow_single_cluster:** Di default *HDBSCAN* non produce un singolo cluster, ma impostando questa variabile a *True*, l'algoritmo produrrà un singolo grande cluster.

Infine, il modello è stato addestrato sfruttando l'implementazione fornita dal modulo `sklearn.cluster`, tenendo traccia degli iperparametri che apportano una configurazione in grado di massimizzare l'efficienza complessiva.

4.5 Support Vector Machine

Kun-Lun Li, Hou-Kuan Huang, Sheng-Feng Tian, Wei Xu all'interno dell'articolo *Improving one-class SVM for anomaly detection* pubblicato nel 2004 da *IEEE*, analizza l'algoritmo *One-Class SVM* per l'*anomaly detection* con ottimi risultati, raggiungendo un *accuracy* del 96% sui dati di test.

Questo ci ha spinto a studiare il comportamento del modello addestrato con l'algoritmo *SVM*, per osservare i risultati e confrontarlo con le altre metodologie descritte all'interno del capitolo.

L'*SVM* (*Support Vector Machine*) è un algoritmo di *ML* supervisionato che classifica i dati trovando una linea o un iperpiano ottimale che massimizza la distanza tra ciascuna classe in uno spazio *N*-dimensionale. Il numero di *feature* nei dati di input determina se l'iperpiano è una linea in uno spazio bidimensionale o un piano in uno spazio *n*-dimensionale. Poiché è possibile trovare più iperpiani per differenziare le classi, massimizzare il margine tra i punti consente all'algoritmo di individuare il miglior confine decisionale tra le classi. Ciò, a sua volta, consente di generalizzare bene i nuovi dati e di fare previsioni di classificazione accurate. Le linee adiacenti all'iperpiano ottimale sono note come vettori di supporto (*Support Vectors*) poiché questi vettori attraversano i punti dati che determinano il margine massimo.

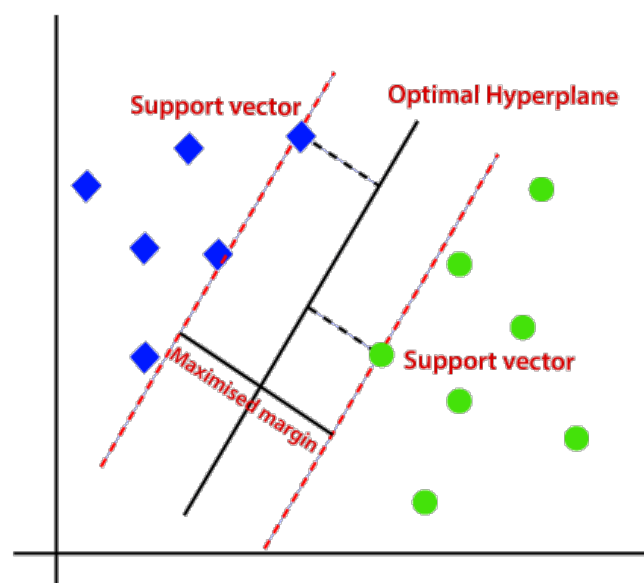


Figura 4.6: Support Vector Machine

Anche in questo caso, è stata definita una griglia di ottimizzazione degli iperparametri:

- **kernel:** tipo di kernel da utilizzare nell'algoritmo scegliendo tra: *poly*, *rbf* e *sigmoid*;
- **gamma:** coefficiente del kernel da specificare per *poly*, *rbf* e *sigmoid*. Definisce l'influenza dei singoli campioni/complessità del confine di decisione;
- **nu:** limite superiore della frazione degli errori di *training* e limite inferiore della frazione dei vettori di supporto. Deve trovarsi all'interno dell'intervallo $(0,1]$.
- **degree:** grado della funzione polinomiale del kernel (utilizzato solo quando *kernel=polynomial*).

Tramite *OneClassSVM* fornito da *sklearn.svm*, viene addestrato il modello utilizzando ogni combinazione presente all'interno della griglia di iperparametri, in modo da massimizzarne le performance.

4.6 Autoencoder

L'approccio iniziale della ricerca si fonda sullo studio di *Janati Idrissi et al.*, nel quale viene impiegata un'architettura basata su *Autoencoder* con notevoli prestazioni. Per tale ragione, l'indagine condotta all'interno del progetto *CLEAR* si è estesa alla sperimentazione del modello generato dall'addestramento mediante *Autoencoder*.

Gli *Autoencoder* sono un sottoinsieme specifico di architetture di *encoder-decoder* addestrate a ricostruire i propri dati di input. Poiché non si basano su dati di addestramento etichettati, gli *Autoencoder* sono considerati un metodo di apprendimento non supervisionato.

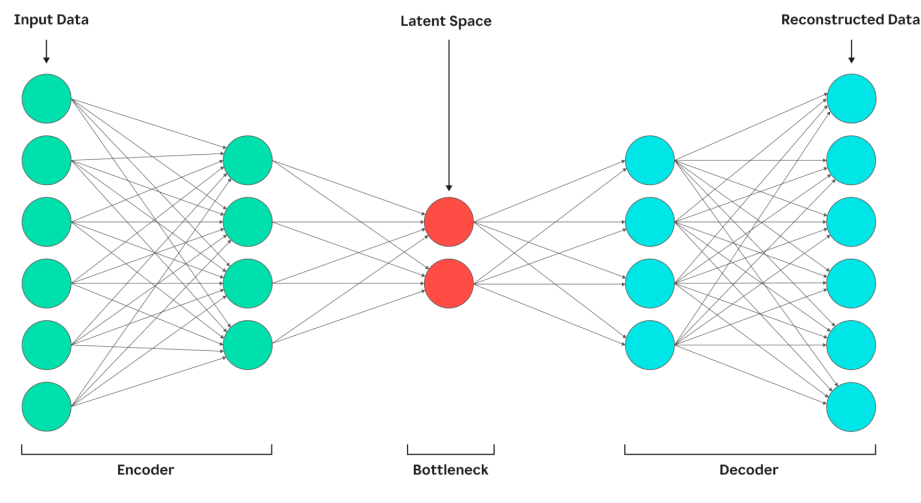


Figura 4.7: Struttura di un Autoencoder

Come mostrato nella figura, *encoder* e *decoder* sono le due componenti simmetriche essenziali per il funzionamento dell'algoritmo.

L'*encoder* comprende strati che codificano una rappresentazione compressa dei dati di input attraverso la riduzione della dimensionalità. In un tipico *Autoencoder*, gli strati nascosti della rete neurale contengono un numero progressivamente inferiore di nodi rispetto allo strato di input: mentre i dati attraversano gli strati dell'*encoder*, vengono compressi riducendo la loro dimensione. I dati di input arrivano all'*encoder* e questi, prima che arrivino al *decoder*, dovranno attraversare un "collo di bottiglia". Questo costringe l'*encoder* a imparare a estrarre e a far passare solo le informazioni più utili per ricostruire con precisione l'input originale.

Il *latent space*, collocato in un "collo di bottiglia" (*bottleneck*), contiene la rappresentazione più compressa dell'input: è sia lo strato di output della rete di *encoder* che lo strato di input della rete di *decoder*. Un obiettivo fondamentale della progettazione e dell'addestramento di un *Autoencoder* è la scoperta del numero minimo di *features* importanti, necessarie per una ricostruzione efficace dei dati di input.

Il *decoder* comprende strati nascosti con un numero progressivamente maggiore di nodi che decomprimono (o decodificano) la rappresentazione codificata dei dati, ricostruendo infine i dati nella loro forma originale precedente alla codifica. Questo output ricostruito viene quindi confrontato con la *ground truth* (i dati veri), che nella maggior parte dei casi è semplicemente l'input originale, per valutare l'efficacia dell'*Autoencoder*. La discrepanza tra l'output e la *ground truth* è chiamata *errore di ricostruzione*.

Lo *script* sviluppato separa il traffico normale da quello anomalo ed effettua una suddivisione dei dati (80% *train*, 20% *validation*) applicata esclusivamente alle istanze rappresentanti il traffico benevolo; la rete neurale, di conseguenza, non riceve mai campioni anomali durante la fase di addestramento. L'architettura simmetrica della rete include un *encoder* che comprime i dati fino a un *collo di bottiglia* centrale mediante livelli `tensorflow.keras.layers.Dense` con funzione di attivazione ReLU, mentre il *decoder* segue una struttura speculare per ricostruire il segnale di partenza. La compilazione del modello impiega l'ottimizzatore Adam e la funzione di perdita *Mean Squared Error (MSE)*.

Al termine dell'addestramento, il *validation set* viene elaborato dall'*Autoencoder* al fine di ottenerne una ricostruzione fedele. Successivamente, viene calcolato lo scarto quadratico medio (MSE) sottraendo punto per punto i dati originali da quelli ricostruiti, elevando al quadrato tali differenze e calcolandone la media sulle quattro *feature* per ciascun campione. Il valore ottenuto viene identificato come `val_mse` ed è impiegato per la definizione della soglia decisionale (*threshold*), calcolata mediante la seguente relazione:

$$threshold = \mu + \sigma$$

μ rappresenta l'errore medio di ricostruzione che il modello commette sul *validation set*. Calcola la media di `val_mse`.

σ è la deviazione standard degli errori, cioè quanto gli errori del traffico normale variano attorno alla media. Sommarla serve ad aggiungere un margine di tolleranza alla media degli errori μ .

La medesima procedura viene quindi applicata al *test set* — composto sia da traffico lecito sia da traffico anomalo — ottenendo il valore `test_mse`.

L'ipotesi sottostante alla stima della soglia sul solo traffico normale risiede nel fatto che la rete, avendo appreso esclusivamente la struttura dei dati leciti, genererà un errore di ricostruzione minimo per questi ultimi. Diversamente, in presenza di traffico anomalo, le difformità tra le *feature* originarie e quelle ricostruite produrranno un MSE significativamente più elevato. La classificazione viene infine eseguita confrontando `test_mse` con la *threshold*: qualora il valore di una riga appartenente a `test_mse`, associato a una richiesta del *client*, superi la soglia stabilita, l'evento viene classificato come un tentativo di attacco.

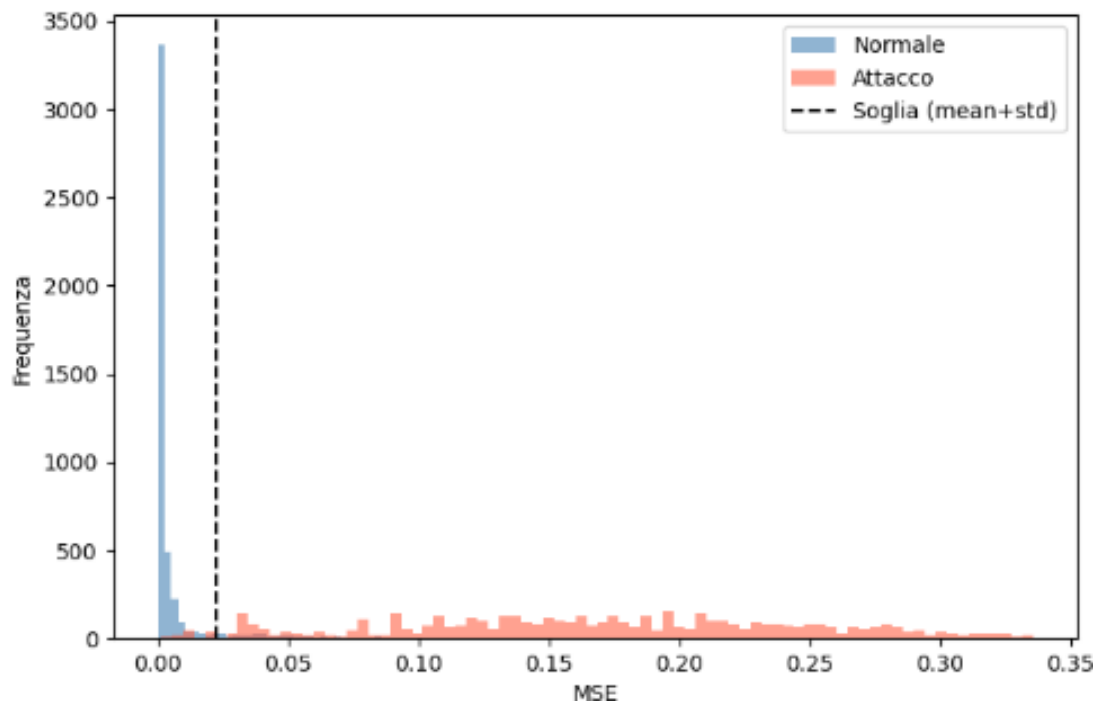


Figura 4.8: Distribuzione degli Errori di Ricostruzione

Il grafico evidenzia la netta separazione, determinata dalla threshold, tra i valori di MSE associati al traffico normale e quelli riferiti al traffico anomalo, mettendo in risalto la sostanziale discrepanza tra le due tipologie di dati.

4.7 Risultati Ottenuti

Ciascuno degli algoritmi considerati raggiunge un *accuracy* intorno all'80% sul *test set* originario. Fa eccezione l'architettura basata su *Autoencoder*, che evidenzia prestazioni nettamente superiori con un'accuratezza pari al 95%. Al fine di valutare la capacità di generalizzazione di ogni modello, è stato predisposto un ulteriore dataset di verifica, caratterizzato dalla medesima struttura del precedente ma popolato con differenti indirizzi *IP* malevoli. I modelli precedentemente addestrati, salvati in formato *.pkl*, sono stati quindi ripristinati per classificare il traffico del nuovo dataset. Il grafico seguente illustra gli esiti di tale analisi comparativa.

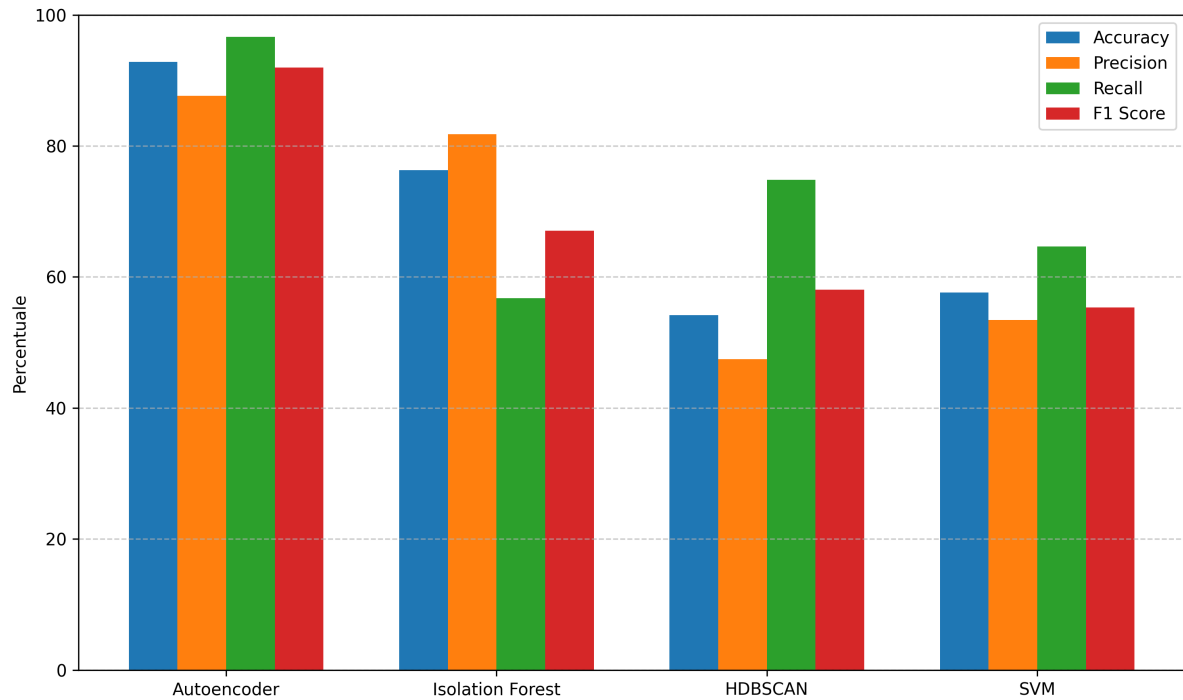


Figura 4.9: Metriche degli algoritmi

Come si evince dai risultati, le prestazioni della maggior parte dei modelli subiscono un drastico calo sul nuovo insieme di dati. Al contrario, l'*Autoencoder* mantiene un'elevata efficacia su tutte le metriche di valutazione, distinguendosi nettamente dalle rimanenti soluzioni. Per tale ragione, gli altri tre approcci sono stati esclusi e l'*Autoencoder* è stato selezionato come modello di riferimento per l'integrazione nell'architettura di *Federated Learning*.

Capitolo 5

Federated Learning

In questo capitolo verrà analizzato l'ultimo *step* del progetto *CLEAR*, che prevede l'addestramento del modello, basato su *Autoencoder*, tramite *Federated Learning*.

Il *Federated Learning* è un processo di *ML* che permette di addestrare un modello di intelligenza artificiale in modo collaborativo e distribuito, senza mai centralizzare o scambiare i dati grezzi degli utenti.

Ci troviamo in un contesto in cui i vari nodi lavorano collettivamente durante il *training* tramite un modello condiviso. La novità sta nel fatto che i dati appartenenti ai singoli nodi, rimangono ai suddetti nodi e non c'è bisogno di scambiarli, mantenendo così la privacy, l'efficienza e la scalabilità. Questa caratteristica lo rende particolarmente adatto a settori in cui la privacy dei dati è fondamentale, come la sanità, la finanza e l'*Internet of Things industriale (IIoT)*.

5.1 Ciclo di vita del FL

Il ciclo di vita di un qualsiasi *FL*, può essere descritto in 6 passaggi fondamentali:

1. **Offerta e selezione dei partecipanti:** il fornitore di servizi pubblica un compito di apprendimento e seleziona i partecipanti in base alle risorse offerte e ai requisiti richiesti;
2. **Trasmissione del modello globale:** il server invia il modello globale iniziale ai partecipanti, specificando le condizioni per l'addestramento;
3. **Addestramento locale:** ogni partecipante aggiorna il modello localmente con i propri dati;

4. **Invio dei parametri:** i client inviano i parametri del modello aggiornati al server. Questi aggiornamenti rappresentano ciò che il modello ha appreso dai dati locali, ma non espongono i dati stessi;
5. **Aggregazione e aggiornamento:** il server aggrega gli aggiornamenti ricevuti per migliorare il modello globale e lo redistribuisce ai partecipanti per nuovi cicli di apprendimento. Per aggregare i pesi dei modelli locali aggiornati dai partecipanti e migliorare il modello globale, viene applicato un algoritmo di aggregazione.
6. **Pubblicazione del modello:** quando il modello raggiunge le prestazioni desiderate, il server conclude l'addestramento ed il modello finale viene pubblicato online per essere utilizzato dagli utenti in applicazioni pratiche.

Il procedimento che va dal punto 2 al punto 5, si ripete molteplici volte e prende il nome di *round*.

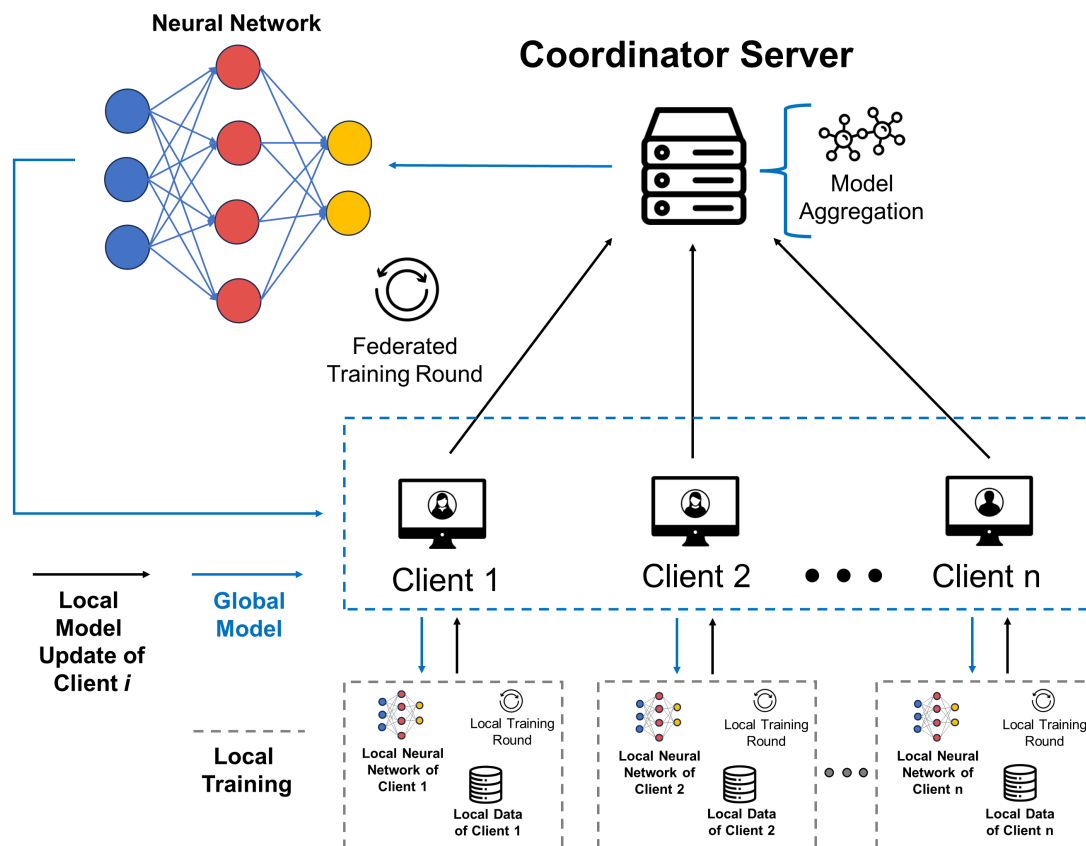


Figura 5.1: Round del FL

5.2 Tipologie di FL

Il *FL* comprende diverse tassonomie, ciascuna caratterizzata da specifiche peculiarità strutturali. In letteratura, tali metodologie vengono prevalentemente classificate in tre categorie principali, in base a come vengono distribuiti i dati, a come scalano i partecipanti e all'architettura di rete utilizzata.

5.2.1 Distribuzione dei dati

In base alle modalità di partizionamento dei dati tra i diversi nodi, il *FL* viene classificato in relazione allo spazio delle caratteristiche misurate (*feature space*) e allo spazio dei campioni (*sample space*) a cui si riferiscono tali caratteristiche. Tale suddivisione dà origine a tre paradigmi fondamentali: *Horizontal Federated Learning (HFL)*, *Vertical Federated Learning (VFL)* e *Federated Transfer Learning (FTL)*.

Tipo di FL	Spiegazione	Esempio
Horizontal Federated Learning	Si verifica quando i set di dati dei diversi partecipanti condividono lo stesso <i>feature space</i> , ma differiscono per i <i>samples</i> . In sostanza, i set di dati hanno la stessa struttura (<i>feature</i>), ma contengono dati relativi a entità diverse.	Due banche in Paesi diversi che usano gli stessi campi per le schede cliente (es. reddito, età), ma con utenti totalmente differenti.
Vertical Federated Learning	Si ha quando i partecipanti dispongono di set di dati che condividono lo stesso <i>sample space</i> , ma differiscono per le <i>features</i> . Ciò significa che ciascun partecipante detiene attributi diversi relativi allo stesso insieme di entità.	Una collaborazione tra una società al dettaglio e una banca, in cui la società al dettaglio possiede i dati di acquisto e la banca possiede i dati finanziari relativi agli stessi clienti.
Federated Transfer Learning	I partecipanti differiscono sia per i soggetti sia per gli attributi, condividendo solo una piccola parte di dati o contesti simili. Sfrutta la conoscenza acquisita da un dominio per migliorare le prestazioni in un altro dominio.	Una collaborazione fra molteplici ospedali, per diagnosticare specifiche malattie, con pazienti (<i>sample spaces</i>) e programmi terapeutici (<i>feature spaces</i>) differenti.

Figura 5.2: HFL vs VFL vs FTL

5.2.2 Architettura di rete

In base alla topologia dell'architettura di rete e alle modalità di coordinamento e aggregazione dei parametri, il *FL* si distingue nei paradigmi *Centralized* e *Decentralized*.

Tipo di FL	Spiegazione	Esempio
Centralized Federated Learning	Prevede un server centrale che orchestra il processo di addestramento. I partecipanti (<i>client</i>) addestrano i modelli locali sui propri dati e inviano i parametri del modello aggiornati al server centrale. Gli aggiornamenti del modello vengono aggregati in un unico punto, semplificando il processo di aggiornamento e rendendolo più facile da gestire.	Un consorzio sanitario in cui gli ospedali addestrano i modelli localmente sui dati dei pazienti e un server centrale aggrega questi modelli per migliorare i modelli di previsione delle malattie.
Decentralized Federated Learning	Elimina la necessità di un server centrale. Al contrario, i partecipanti comunicano direttamente tra loro per condividere e aggregare gli aggiornamenti del modello. Questa comunicazione <i>peer-to-peer</i> garantisce l'assenza di un singolo punto di fallimento (<i>single point of failure</i>) e migliora la privacy distribuendo il processo di aggregazione.	Una rete di dispositivi <i>mobile</i> che addestra collaborativamente un modello per prevedere i pattern di utilizzo delle app senza affidarsi a un server centrale.

Figura 5.3: CFL vs DFL

5.2.3 Scala dei partecipanti

In base alla natura dei nodi partecipanti, alla topologia di rete e alla scala di distribuzione, il *FL* si suddivide principalmente in due architetture applicative: *Cross-Silo* e *Cross-Device*.

Tipo di FL	Spiegazione	Esempio
Cross-Silo Federated Learning	Coinvolge un numero ridotto di partecipanti (<i>silo</i>) affidabili e stabili, come organizzazioni o istituzioni. Questi silo dispongono in genere di notevoli risorse computazionali e di connessioni di rete stabili.	Università che collaborano a un progetto di ricerca in cui ciascun ateneo possiede il proprio set di dati, ma intende costruire un modello predittivo comune.
Cross-Device Federated Learning	Coinvolge un numero elevato di dispositivi come <i>smartphone</i> , dispositivi <i>IoT</i> o dispositivi <i>edge</i> . Questi dispositivi sono in genere meno affidabili e presentano capacità computazionali variabili e una connettività di rete intermittente.	L'addestramento di un modello di inserimento del testo predittivo su dispositivi <i>mobile</i> , in cui ciascun dispositivo contribuisce al modello senza inviare i dati dell'utente a un server centrale.

Figura 5.4: Cross-Silo FL vs Cross-Device FL

5.3 Script Python

L'ambiente di test prevede l'impiego di tre nodi, ciascuno dei quali dispone di un proprio dataset. I dati mantengono la medesima struttura analizzata nel Capitolo 3 e impiegata per l'addestramento dei modelli nel Capitolo 4, basandosi sui file di *log* della piattaforma *Sardegna Solidale*. Ciascun nodo presenta tuttavia un insieme distinto di indirizzi *IP* associati ai client malevoli coinvolti nell'attacco *Brute Force*.

Lo *script*, completata la fase di *Data Preprocessing*, definisce le variabili globali del sistema, istanzia la struttura dell'*Autoencoder* (coerente con l'architettura

descritta nel capitolo precedente) per ciascuno dei tre nodi e implementa la logica di *FL* mediante *Flower*, un *framework open-source* in ambiente *Python*. Viene definita la classe `AutoencoderClient`, la quale estende la classe base `fl.client.NumPyClient` fornita da *Flower* (importata con l'alias `import flwr as fl`). Tale classe costituisce un'interfaccia standard che consente di gestire un client federato operando direttamente sugli *array NumPy* associati ai pesi del modello, astenendo lo sviluppatore dalla gestione manuale della serializzazione/deserializzazione a basso livello del protocollo di comunicazione. La classe `AutoencoderClient` implementa diversi metodi fondamentali, tra cui il metodo `fit`, centrale nel processo di apprendimento:

1. Riceve i pesi globali trasmessi dal server e li carica nel modello locale;
2. Addestra il modello locale esclusivamente sui campioni appartenenti al traffico lecito;
3. Restituisce i pesi aggiornati insieme al numero di esempi usati. Questi sono i parametri che il server andrà ad utilizzare per calcolare la media pesata.

La funzione principale (`main`) avvia la procedura iterativa di *FL*:

1. **Broadcast:** il server invia i pesi casuali del modello globale a tutti i client selezionati come base di partenza. Ciò che conta, ai fini della correttezza del *FL*, non è che valore abbiano questi pesi iniziali, ma che siano gli stessi per tutti i client così da garantire che tutti i client partano dallo stesso identico punto nello spazio dei parametri, anche se quel punto è casuale;
2. **Addestramento locale:** i 3 client, definiti dalla classe `AutoencoderClient`, eseguono l'addestramento locale del modello per un numero predefinito di epoche;
3. **Upload:** ogni client rimanda al server i pesi aggiornati ed il numero di campioni utilizzati.
4. **Aggregazione:** i pesi vengono aggregati dal server tramite la strategia del *FedAvg*. Quest'ultimo è l'algoritmo di aggregazione più comune nel *FL* e si limita a fare una media pesata dei pesi ricevuti dai vari client per costruire un modello globale via via migliore. La formula matematica utilizzata è la seguente:

$$w_{t+1} = \sum_{k=1}^K \frac{n_k}{N} w_t^k$$

dove:

- w_{t+1} : indica i pesi del modello globale al *round* $t + 1$;
- K : rappresenta il numero totale di client partecipanti;
- n_k : indica la quantità di campioni appartenenti al *dataset* del *client* k ;
- N : corrisponde al numero totale di campioni;
- w_t^k : rappresenta l'insieme dei pesi locali aggiornati dal *client* k al termine del *round* t .

5. **Valutazione:** viene chiamata la funzione *evaluate_aggregated* sul modello appena aggregato, per misurare le performance globali.

La procedura descritta viene iterata per un numero prefissato di *round*.

5.4 Risultati Ottenuti

Il processo di addestramento è stato iterato per un totale di tre *round*. Tale scelta è giustificata dal fatto che il modello converge rapidamente verso prestazioni ottimali già nelle fasi iniziali; oltre tale soglia, l'andamento delle metriche si stabilizza, rendendo superfluo un prolungamento dell'addestramento.

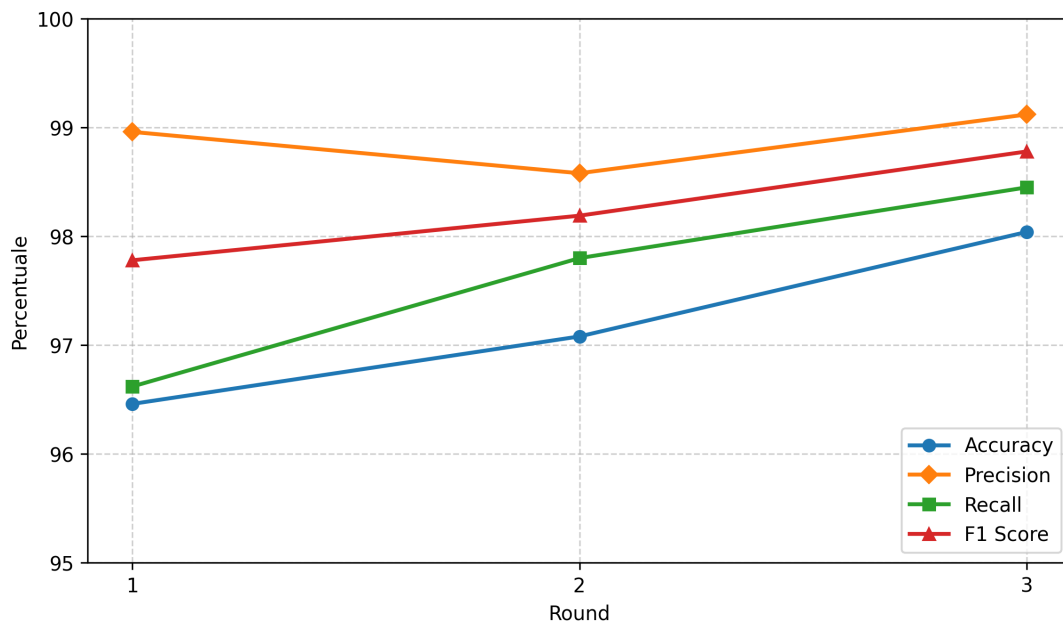


Figura 5.5: Metriche Globali nei Round di FL

Il grafico illustra l'evoluzione delle prestazioni del modello globale nel corso dei tre *round*.

Le metriche considerate sono le medesime analizzate nel capitolo precedente, ovvero *Accuracy*, *Precision*, *Recall* e *F1-Score*.

I codici sorgente sviluppati nel corso della ricerca — dalla configurazione di *Fluent Bit* ai moduli relativi all'addestramento dell'*Autoencoder* e all'architettura di *Federated Learning* — sono liberamente consultabili al seguente *repository* GitHub: *CLEAR*.

Capitolo 6

Sviluppi Futuri

Il progetto *CLEAR* ha consentito di raggiungere gli obiettivi definiti all'inizio della ricerca, strutturando una metodologia sistematica per l'analisi del problema fino al completamento dell'addestramento del modello mediante *FL*. Le prestazioni finali registrano valori di assoluto rilievo: 98.04% di *Accuracy*, 99.12% di *Precision*, 98.45% di *Recall* e 98.78% di *F1-score*.

Il caso di studio proposto, si è collocato in un ambiente di sperimentazione controllato caratterizzato dall'analisi di traffico lecito combinato a una singola tipologia di minaccia, ovvero l'attacco di tipo *Brute Force*. Tale approccio ha risposto alla volontà di adottare un modello di sviluppo incrementale, validando inizialmente l'architettura in un contesto contenuto prima di introdurre un maggior livello di complessità.

Sulla base dell'infrastruttura di *CLEAR* e delle elevate prestazioni ottenute, i principali sviluppi futuri risiedono nell'estensione del profilo di attacco considerato. A fronte dell'introduzione di nuove categorie di minacce, si procederà al riaddestramento del modello monitorandone i risultati. Qualora si osservi un degrado nelle metriche di valutazione, si renderà necessario un affinamento della pipeline di pre-elaborazione, intervenendo sullo *script lua* di *Fluent Bit* per l'estrazione di ulteriori *feature* maggiormente discriminanti.

Concluso l'addestramento del modello, basato su differenti tipologie di attacchi, si prevede l'esecuzione di test in ambienti operativi reali. La visione applicativa di *CLEAR* mira infatti all'integrazione all'interno dell'infrastruttura di una *Smart City*.

In tale contesto urbano, caratterizzato da una fitta rete di sensori, dispositivi *IoT* e sistemi digitali interconnessi (quali mobilità intelligente, reti energetiche e videosorveglianza), ciascun dominio genera un flusso continuo di dati verso i propri *gateway* locali, per automatizzare decisioni, con l'obiettivo di rendere i servizi urbani più efficienti, sostenibili e reattivi.

L'obiettivo di *CLEAR* è quello di posizionarsi all'interno di quest'architettura distribuita in qualità di *NIDS*, classificando e mitigando le minacce informatiche in tempo reale.

Bibliografia

- [1] Kun-Lun Li et al. *Improving one-class SVM for anomaly detection*. 2004. <https://ieeexplore.ieee.org/abstract/document/1260106>.
- [2] Meryem Janati Idrissi et al. *Fed-ANIDS: Federated learning for anomaly-based network intrusion detection systems*. 2023. <https://www.sciencedirect.com/science/article/pii/S0957417423015026>.
- [3] Wilson Chua et al. *Web Traffic Anomaly Detection Using Isolation Forest*. 2024. <https://www.mdpi.com/2227-9709/11/4/83>.
- [4] Rawand Raouf Abdalla e Alaa Khalil Jumaa. *Log File Analysis Based on Machine Learning: A Survey*. 2022. <https://journals.uhd.edu.iq/index.php/uhdjst/article/view/994>.
- [5] Checkpoint. *Definizione del Denial of Service*. <https://www.checkpoint.com/it/cyber-hub/cyber-security/what-is-denial-of-service/>.
- [6] Canadian Institute for Cybersecurity. *Come funziona CICFlowMeter*. <https://www.unb.ca/cic/research/applications.html>.
- [7] Canadian Institute for Cybersecurity. *Spiegazione del dataset CIC IoT-DIAD2024*. <https://www.unb.ca/cic/datasets/iot-diad-2024.html>.
- [8] Canadian Institute for Cybersecurity. *Spiegazione del dataset CIC-DDoS2019*. <https://www.unb.ca/cic/datasets/ddos-2019.html>.
- [9] Canadian Institute for Cybersecurity. *Spiegazione del dataset CIC-IDS2017*. <https://www.unb.ca/cic/datasets/ids-2017.html>.
- [10] Canadian Institute for Cybersecurity. *Spiegazione del dataset CSE-CIC-IDS2018*. <https://www.unb.ca/cic/datasets/ids-2018.html>.
- [11] Denodo. *Cos'è e perchè il Data Preprocessing è così importante*: <https://www.denodo.com/en/glossary/data-preprocessing-definition-importance-key-techniques>.

- [12] AleksandrS.Baklashov e DmitryS.Kulyabov. *Statistical and density-based clustering techniques in the context of anomaly detection in network systems: A comparative analysis*. 2025. <https://cyberleninka.ru/article/n/statistical-and-density-based-clustering-techniques-in-the-context-of-anomaly-detection-in-network-systems-a-comparative-analysis>.
- [13] Elastic. *Introduzione a Logstash*. <https://www.elastic.co/logstash>.
- [14] Fluentbit. *Documentazione di Fluent Bit*. <https://fluentbit.io>.
- [15] Fluentd. *Introduzione a Fluentd*. <https://www.fluentd.org/architecture>.
- [16] GeeksforGeeks. *Approfondimento al Federated Learning e alle sue differenti tipologie*. <https://www.geeksforgeeks.org/machine-learning/types-of-federated-learning-in-machine-learning/>.
- [17] GeeksforGeeks. *Definizione dell'algoritmo Isolation Forest*. <https://www.geeksforgeeks.org/machine-learning/what-is-isolation-forest/>.
- [18] HPE. *Approfondimento dei sistemi IDS e IPS*. <https://www.hpe.com/it/it/what-is/ids-ips.html>.
- [19] IBM. *Cos'è e com'è fatta la struttura di un Autoencoder*. <https://www.ibm.com/it-it/think/topics/autoencoder>.
- [20] IBM. *Cos'è una rete neurale*. <https://www.ibm.com/it-it/think/topics/neural-networks>.
- [21] IBM. *Definizione dei parametri di un modello di IA*. <https://www.ibm.com/it-it/think/topics/model-parameters>.
- [22] IBM. *Definizione del Decision Tree*. <https://www.ibm.com/it-it/think/topics/decision-trees>.
- [23] IBM. *Definizione dell'algoritmo Random Forest*. <https://www.ibm.com/it-it/think/topics/random-forest>.
- [24] IBM. *Definizione dell'algoritmo SVM*. <https://www.ibm.com/it-it/think/topics/support-vector-machine>.
- [25] labelf.ai. *Definizione di Accuracy, Precision, Recall ed F1-Score*. <https://labelf.ai/blog/what-is-accuracy-precision-recall-and-f1-score>.
- [26] Pulp Learning. *Cosa sono i set di Training, Validation e Test*. <https://pulplearning.altervista.org/train-validation-test-cosa-sono-e-come-si-usano-nel-machine-learning/>.
- [27] Lua. *Cos'è e perchè utilizzare il linguaggio di scripting Lua*. <https://www.lua.org/about.html>.

- [28] Medium. *Confronto tra Fluentd, Fluent Bit e Logstash*. <https://sarankamandula.medium.com/a-quick-glance-into-logstash-fluentd-and-fluentbit-f44491f98823>.
- [29] Medium. *Confronto tra Suricata e Snort*. <https://medium.com/@redfanatic7/suricata-vs-snort-detailed-guide-to-the-programs-c331cff452a1>.
- [30] Medium. *Perchè conviene utilizzare la tecnica del Min-Max-Scaler*. <https://medium.com/@iamkamleshurangi/how-min-max-scaler-works-9fbeb9347da>.
- [31] Proofpoint: *Definizione del Brute Force attack*. <https://www.proofpoint.com/it/threat-reference/brute-force-attack>.
- [32] Malika Addou e Fatimazahra Barramou Said Ouiazzane. *A Suricata and Machine Learning Based Hybrid Network Intrusion Detection System*. 2022. https://link.springer.com/chapter/10.1007/978-3-030-91738-8_43.
- [33] ScienceDirect. *Approfondimento del sistema NIDS*. <https://www.sciencedirect.com/topics/computer-science/network-intrusion-detection-system>.
- [34] ScienceDirect. *Panoramica sul Federated Learning*. <https://www.sciencedirect.com/science/article/pii/S2405844024141680>.
- [35] Scikit-learn. *Analisi dei parametri appartenenti alla classe sklearn.cluster.HDBSCAN*. <https://scikit-learn.org/stable/modules/generated/sklearn.cluster.HDBSCAN.html>.
- [36] Scikit-learn. *Analisi dei parametri appartenenti alla classe sklearn.ensemble.IsolationForest*. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html>.
- [37] Scikit-learn. *Analisi dei parametri appartenenti alla classe sklearn.svm.OneClassSVM*. <https://scikit-learn.org/stable/modules/generated/sklearn.svm.OneClassSVM.html>.
- [38] Snort. *Guida al sistema Open Source Snort*. <https://www.snort.org/>.
- [39] Suricata. *Guida al sistema Open Source Suricata*. <https://docs.suricata.io/en/suricata-8.0.5/index.html>.
- [40] Zeek. *Guida al sistema Open Source Zeek*. <https://docs.zeek.org/en/master/about/what.html>.

Ringraziamenti

Desidero esprimere la mia più profonda gratitudine al mio correlatore, il professor Andrea Borghesi, per avermi aiutato con la correzione dell'elaborato e per essere sempre stato tempestivo e preciso con le sue risposte. È come se fossi stato il mio relatore.

Ringrazio l'azienda Net Service per avermi dato l'opportunità di svolgere quest'esperienza di tirocinio, la quale mi ha portato ad avvicinarmi con nuove tematiche e ad integrarmi all'interno del progetto di ricerca CLEAR. Ringrazio tutti i colleghi che ho conosciuto per avermi dato una mano e avermi reso quest'esperienza ancora più piacevole fra chiacchiere, pranzi e pause caffè.

Un ringraziamento speciale va ai miei genitori, per aver contribuito al raggiungimento di questo mio secondo traguardo e per aver creduto ancora una volta in me. Mi dispiace se questo periodo è durato più del previsto, ma questo mi fa apprezzare ancora di più la vostra pazienza. Inoltre vi ringrazio per avermi dato la possibilità di intraprendere questa nuova esperienza di studio in una città lontana da casa.

Un'altra persona molto importante che desidero ringraziare è la mia fidanzata Elisa. Grazie per aver creduto in me come fai sempre, anche più di quanto lo faccia io. Nonostante la distanza mi sei stata vicino e ci sei sempre stata per ascoltarmi e darmi consigli. Aspetto con ansia questo giorno, soprattutto per poterti rivedere e per festeggiare come sappiamo fare noi...mangiando come cinghialetti per tutta Bologna.

Ringrazio tutti i miei amici storici per essere rimasti ancora al mio fianco nel corso di tutti questi anni e per avermi fatto vivere numerosi momenti che ricorderò per sempre. Anche una semplice giornata passata con voi è sempre importante per me. Un ringraziamento in particolare lo voglio dedicare ad Alessio. Ultimamente ci siamo riavvicinati molto e mi sei stato vicino dimostrandomi quanto tieni a me. Sappi che ho apprezzato tutto questo e non vedo l'ora di pisciarmi addosso con te.

Ringrazio i miei compagni di corso, per il supporto che mi hanno dato e per le lezioni passate in compagnia a cazzeggiare. Un grazie va anche ai coinquilini e a tutti gli altri amici che ho conosciuto qui a Bologna, perchè è per merito vostro che ho vissuto nuove esperienze indimenticabili. Sono contento di aver conosciuto delle persone come voi.